



ROYAUME DU MAROC

المملكة المغربية

,Ministère de l'Enseignement Supérieur,
de la Formation des Cadres et de la Recherche Scientifique

CONCOURS NATIONAL COMMUN

d'Admission dans les Établissements de Formation

d'Ingénieurs et Établissements Assimilés

Édition 2021 - 2015

ÉPREUVE D'INFORMATIQUE

Partie : Programmation python

Préparation CNC : Informatique

Filières : **MP/PSI/TSI**

Durée **2 heures**

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Remarques générales :

- ❖ Cette épreuve est composée d'un exercice et de trois parties tous indépendants ;
- ❖ Toutes les instructions et les fonctions demandées seront écrites en Python ;
- ❖ Les questions non traitées peuvent être admises pour aborder les questions ultérieures ;
- ❖ Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

~~~~~

**Important :** Le candidat doit impérativement traiter l'exercice ci-dessous, et inclure les réponses dans les premières pages du cahier de réponse.

**Exercice :** (4 points)

***Développement limité du cosinus***

Le développement limité de  $\cos(x)$ , à l'ordre  $n$ , est :

$$\cos(x) = \sum_{i=0}^n (-1)^i * \frac{x^{2i}}{(2i)!}$$

- 1 pt Q1- Écrire la fonction **factoriel** (**k**), qui reçoit en paramètre un entier positif **k**, et qui retourne la valeur de factorielle **k** :  $k! = 1 * 2 * 3 * \dots * (k-1) * k$ .

**NB :** La fonction **factoriel** (0) retourne 1

- 1 pt Q2- Écrire la fonction **calcul** (**x**, **k**) qui reçoit en paramètres un réel **x** et un entier positif **k**, et qui retourne la valeur de l'expression suivante :

$$(-1)^k * \frac{x^{2k}}{(2k)!}$$

- 0.5 pt Q3- Déterminer la complexité de la fonction **calcul** (**x**, **k**), et justifier votre réponse.

- 1.5 pt Q4- Écrire la fonction **cosinus** (**x**, **n**) qui reçoit en paramètres un réel **x** et un entier positif **n**, et qui retourne la valeur du développement limité de  $\cos(x)$ , à l'ordre **n**.

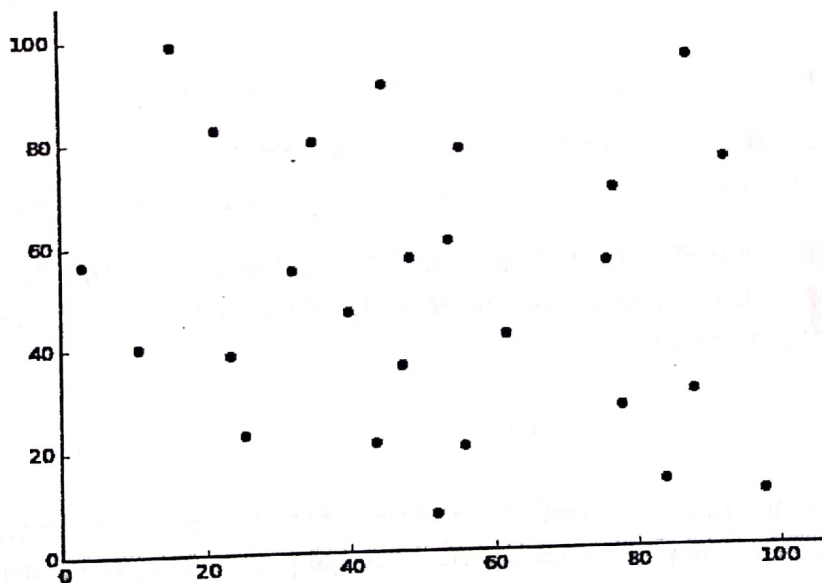
## Partie III : Problème Python

### Géométrie algorithmique

#### *Recherche des deux points les plus rapprochés*

La **géométrie algorithmique** est un domaine qui traite des algorithmes manipulant des concepts géométriques. La discipline qui a le plus contribué historiquement au développement de la géométrie algorithmique est l'infographie. Toutefois, à l'heure actuelle, la géométrie algorithmique se voit fréquemment impliquée dans des problèmes d'algorithmique générale. Aujourd'hui, la géométrie algorithmique est utilisée en infographie, en robotique, pour les systèmes d'information géographique, ...

En géométrie algorithmique, la **recherche des deux points les plus rapprochés** est le problème qui consiste à trouver une paire de points, d'un ensemble fini de points dans un espace métrique, dont la distance est minimale. Il fait partie des problèmes fondateurs de la géométrie algorithmique.



Ce problème de recherche voit son utilité dans les transports aériens ou maritimes par exemple. Il est utilisé par les contrôleurs aériens pour repérer les avions les plus proches les uns des autres (l'espace considéré ici est à 3 dimensions), et ainsi prévenir le risque de collision.

Dans cette partie, nous allons nous intéresser au problème suivant : étant donné un ensemble de points du plan, on cherchera à trouver le couple de points les plus rapprochés au sens de la distance euclidienne.

Un point du plan sera représenté par un tuple de réels  $(x, y)$  représentant ses coordonnées dans un repère orthonormé. L'ensemble des points considérés sera quant à lui stocké dans une liste  $P$ .

Nous supposons que la liste  $P$  contient au moins deux points, et que les points de  $P$  sont tous différents deux à deux. Nous supposons aussi que la liste  $P$  contient une seule paire de points, à distance minimale.



**Exemple :**

Les points de la figure précédente sont représentés par la liste **P** suivante :

$P = [ (84.04, 12.68), (21.23, 82.75), (35.00, 80.00), (78.00, 27.00), (52.01, 7.03), (48.59, 56.77), (88.16, 30.04), (77.3, 69.82), (40.00, 46.46), (10.72, 39.91), (97.98, 10.52), (31.86, 54.75), (54.19, 60.09), (87.97, 95.69), (23.21, 38.41), (93.19, 75.47), (55.82, 78.18), (25.21, 22.75), \dots ]$

NB : Dans tous les exemples de cette partie, on utilisera cette liste **P**.

**Distance euclidienne entre deux points :**

La distance euclidienne entre deux points **A** et **B** de coordonnées  $(x_A, y_A)$  et  $(x_B, y_B)$ , est calculée par la formule suivante :

$$\sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

**Q.1-** Écrire la fonction *distance* (**A**, **B**) qui reçoit en paramètres deux tuples **A** et **B** qui représentent deux points. La fonction retourne la valeur de la distance euclidienne entre les deux points **A** et **B**.

**Exemple :**

**A** = (84.04, 12.68) et **B** = (21.23, 82.75)

La fonction *distance* (**A**, **B**) retourne le nombre : 94.10048352691925

On suppose que la complexité temporelle de la fonction racine carrée est constante. Dans la suite de cette partie, nous nous intéresserons à la mise au point de deux algorithmes différents, qui permettent de trouver les deux points les plus rapprochés, dans la liste **P**.

**A- Algorithme naïf :**

Le premier algorithme est un algorithme naïf qui consiste à faire une recherche exhaustive : considérer successivement tous les couples de points dans la liste **P**, ce qui peut se faire facilement à l'aide de deux boucles imbriquées.

**Q.2-** Écrire la fonction *plus\_proches* (**P**) qui reçoit en paramètre une liste de points **P**. En utilisant le principe de l'algorithme naïf précédent, la fonction retourne deux tuples qui représentent les deux points les plus rapprochés dans la liste **P**.

**Exemple :**

La fonction *plus\_proche* (**P**) retourne les deux points : (48.59, 56.77) , (54.19, 60.09)

**Q.3-** Déterminer la complexité de la fonction *plus\_proches* (**P**).

Justifier votre réponse.

**B- Algorithme efficace**



Le deuxième algorithme est basé sur le paradigme : *diviser pour régner*. Cet algorithme nécessite de disposer de deux copies de la liste des points  $P$  : l'une triée dans l'ordre croissant des abscisses des points, et l'autre triée dans l'ordre croissant des ordonnées des points.

**Q.4-** Écrire la fonction `tri_points` ( $P, k$ ) qui reçoit en paramètre la liste  $P$ , et  $k$  un entier tel que :  $k \in \{0, 1\}$ . La fonction retourne une copie de la liste  $P$ , triée dans l'ordre croissant du  $k^{\text{ème}}$  élément de chaque tuple de la liste  $P$ .

**Exemples :**

- La fonction `tri_points` ( $P, 0$ ) retourne la liste triée selon les abscisses des points :  
[(2.99, 56.44), (10.72, 39.91), (15.33, 99.03), (21.23, 82.75), (23.21, 38.41), ... ]
- La fonction `tri_points` ( $P, 1$ ) retourne la liste triée selon les ordonnées des points :  
[(52.01, 7.03), (97.98, 10.52), (84.04, 12.68), (56.0, 20.0), (43.7, 20.82), ... ]

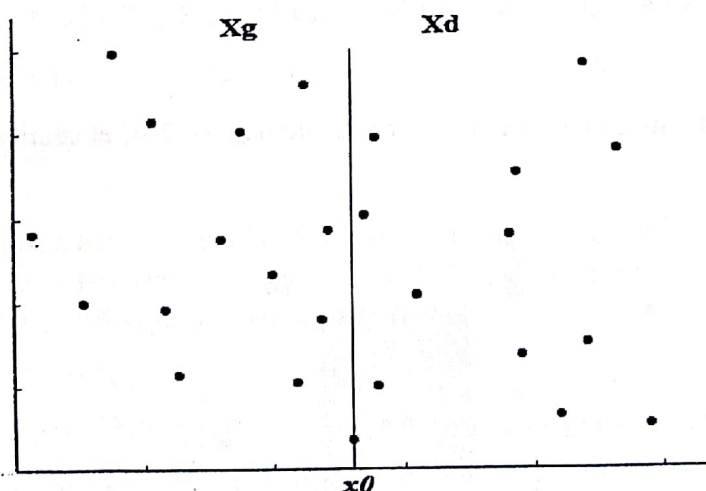
On suppose que la liste  $X$  est la copie de  $P$ , triée dans l'ordre croissant des abscisses des points.

On considère deux variables  $n$  et  $x_0$ , telles que :

- $n \leftarrow$  Longueur de  $X$  ;
- $x_0 \leftarrow$  l'abscisse du point au milieu de la liste  $X$ , à l'indice  $n/2$ .

Ensuite, à partir des éléments de la liste triée  $X$ , on crée deux listes  $X_g$  et  $X_d$ , telles que :

- $X_g$  contient la première moitié de la liste  $X$  : les éléments de  $X$  d'indice  $i$  tel que  $0 \leq i < n/2$  ;
- $X_d$  contient la deuxième moitié de la liste  $X$  : les éléments de  $X$  d'indice  $i$  tel que  $n/2 \leq i < n$ .



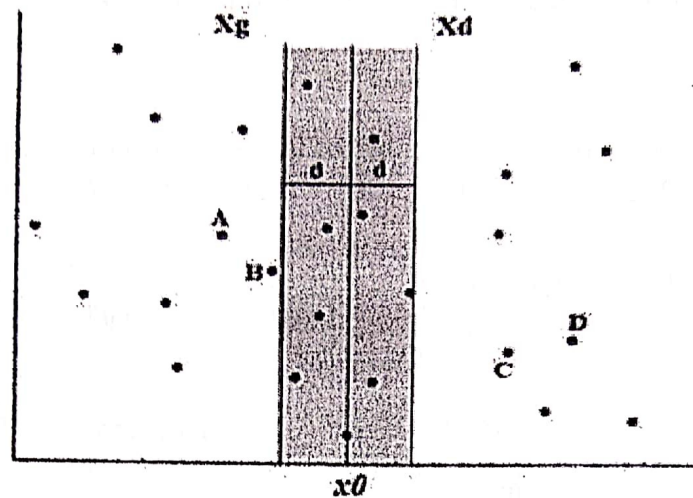
Supposons que  $A$  et  $B$  sont les deux points les plus rapprochés dans la liste  $X_g$ , et que  $d_1$  est la distance entre  $A$  et  $B$ .

Supposons aussi que  $C$  et  $D$  sont les deux points les plus rapprochés dans la liste  $X_d$ , et que  $d_2$  est la distance entre  $C$  et  $D$ .

On pose :  $d \leftarrow$  Le minimum de  $d_1$  et  $d_2$

Si deux points de la liste  $P$  sont à une distance strictement inférieure à  $d$ , alors forcément :

- L'un des deux points appartient à la liste  $X_g$ , et l'autre appartient à la liste  $X_d$  ;
- Et les deux points se trouvent dans la bande verticale qui contient les points d'abscisses comprises entre  $x_0-d$  et  $x_0+d$ .



Q.5- Écrire la fonction `bande_verticale` ( $Y, x_0, d$ ) qui reçoit en paramètres la liste  $Y$  des points triés dans l'ordre croissant des ordonnées, et les deux réels  $x_0$  et  $d$ . La fonction retourne la liste  $B$  des points de  $Y$  dont les abscisses sont comprises entre  $x_0-d$  et  $x_0+d$ , et triés dans l'ordre croissant des ordonnées.

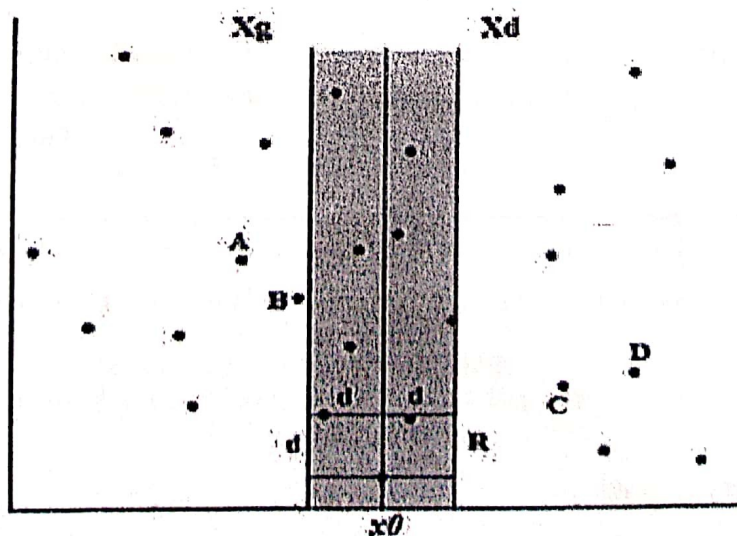
Exemple :

$Y = [(52.01, 7.03), (97.98, 10.52), (84.04, 12.68), (56.0, 20.0), (43.7, 20.82), (25.21, 22.75), \dots]$

On suppose que  $x_0 = 52.01$  et  $d = 10.605055398252285$

La fonction `bande_verticale` ( $Y, x_0, d$ ) retourne la liste  $B$  suivante :  $[(52.01, 7.03), (56.0, 20.0), (43.7, 20.82), (47.54, 35.79), (62.0, 41.5), (48.59, 56.77), (54.19, 60.09), (55.82, 78.18), (45.0, 91.0)]$

La liste  $B$  contient les points situés dans la bande verticale de largeur  $2*d$ , et centrée en la droite verticale passant par l'abscisse  $x_0$ .





Supposons qu'ils existent deux points  $B_i$  et  $B_j$  de la liste  $B$ , tels que la distance entre ces deux points est inférieure à  $d$ . Les points  $B_i$  et  $B_j$  sont donc dans un rectangle  $R$  de dimension  $d \times 2d$  centré en la droite verticale passant par  $x_0$ . On suppose que le point  $B_i$  est sur le côté inférieur du rectangle  $R$ .

Montrons que le rectangle  $R$  contient au plus 8 points de  $B$ . Pour cela, on considère le carré de dimension  $d \times d$  qui forme la moitié gauche du rectangle  $R$  :

- Puisque tous les points de  $X_g$  sont distants d'au moins  $d$ , il y en a au plus 4 points qui se trouvent dans ce carré : 1 point par sous-carré de dimension  $(d/2) \times (d/2)$  de diagonale  $d\sqrt{2}/2 < d$ .
- De même, il y a au plus 4 points qui se trouvent dans la moitié droite de  $R$ .

Donc, le rectangle  $R$  contient au plus 8 points de  $B$ , qui sont classés par ordonnées croissantes, et on a bien :  $i < j \leq i+7$  et la distance entre les points  $B_i$  et  $B_j$  est inférieure à  $d$ .

Ainsi, on déduit que si deux points de  $B$  sont à une distance inférieure à  $d$ , alors ils sont situés à au plus 7 positions l'un de l'autre dans la liste  $B$ .

**Q.7-** Écrire la fonction de complexité linéaire *plus\_proches\_bande* ( $B$ ) qui reçoit en paramètre la liste  $B$  qui contient les points de la bande verticale. La fonction retourne les deux points les plus rapprochés dans la liste  $B$ .

**Exemple :**

$B = [(52.01, 7.03), (56.0, 20.0), (43.7, 20.82), (47.54, 35.79), (62.0, 41.5), (48.59, 56.77), (54.19, 60.09), (55.82, 78.18), (45.0, 91.0)]$

La fonction *plus\_proches\_bande* ( $B$ ) retourne les deux points : (48.59 , 56.77) , (54.19 , 60.09)

**Q.8-** Justifier que la fonction *plus\_proches\_bande* ( $B$ ) est de complexité linéaire.

En utilisant les fonctions précédentes, on peut mettre en point un algorithme récursif permettant de trouver les deux points les plus rapprochés à partir des deux listes  $X$  (liste des points triés par abscisses croissantes) et  $Y$  (liste des points triés par ordonnées croissantes) :

- $n \leftarrow$  longueur de la liste  $X$
- Si  $n \leq 3$  alors retourner les deux points les plus rapprochés, en utilisant l'algorithme naïf ;
- $x_0 \leftarrow$  l'abscisse du point au milieu de la liste  $X$
- Créer deux listes  $X_g$  et  $X_d$ , qui contiennent respectivement la première moitié et la deuxième moitié de la liste  $X$  ;
- Avec un appel récursif, déterminer les deux points  $A$  et  $B$  les plus rapprochés dans  $X_g$  ;
- $d_1 \leftarrow$  la distance entre  $A$  et  $B$  ;
- Avec un appel récursif, déterminer les deux points  $C$  et  $D$  les plus rapprochés dans  $X_d$  ;



- h.  $d2 \leftarrow$  la distance entre C et D ;
- i. Déterminer la paire de points (E, F) les plus rapprochés parmi les paires (A, B) et (C, D), ainsi que la distance d entre E et F ;
- j. Déterminer la liste B qui représente la bande verticale de largeur  $2*d$ , centrée en la droite verticale passant par  $x0$  ;
- k. Si la longueur de la liste B est inférieure strictement à 2, alors on retourne la paire de points (E, F) ;
- l. Si la liste B contient au moins deux points, alors déterminer les deux points P et Q les plus rapprochés dans B, ainsi que la distance  $d3$  entre P et Q ;
- m. Retourner la paire de points à distance minimale parmi les paires (E, F) et (P, Q)

Q.9- Écrire la fonction récursive *plus\_proches\_rec* (X, Y) qui reçoit en paramètres les deux listes X et Y, et qui retourne les deux points les plus rapprochés.

Exemple :

X = [(2.99, 56.44), (10.72, 39.91), (15.33, 99.03), (21.23, 82.75), (23.21, 38.41), ... ]

Y = [(52.01, 7.03), (97.98, 10.52), (84.04, 12.68), (56.0, 20.0), (43.7, 20.82), ... ]

La fonction *plus\_proches\_rec* (X, Y) retourne les points : (48.59, 56.77) , (54.19, 60.09)

Q.10- Déterminer la complexité de la fonction *plus\_proches\_rec* (X, Y), et justifier votre réponse.

===== FIN DE L'ÉPREUVE =====

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

**Remarques générales :**

- ✓ Cette épreuve est composée d'un exercice et de trois parties tous indépendants ;
- ✓ Toutes les instructions et les fonctions demandées seront écrites en Python ;
- ✓ Les questions non traitées peuvent être admises pour aborder les questions ultérieures ;
- ✓ Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

~~~~~

Important : Le candidat doit impérativement traiter l'exercice ci-dessous, et inclure les réponses dans les premières pages du cahier de réponse.

Exercice : (4 points)

Développement limité de 'exponentielle'

Le développement limité de e^x , à l'ordre n , est :

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!}$$

1 pt Q1- Écrire la fonction **factoriel** (k) qui reçoit en paramètre un entier positif k , et qui retourne la valeur de factorielle k : $k! = 1 * 2 * 3 * \dots * (k-1) * k$.

NB : La fonction **factoriel** (0) retourne 1

0.75 pt Q2- Déterminer la complexité de la fonction **factoriel** (k), et justifier votre réponse.

0.75 pt Q3- Écrire la fonction **calcul** (x , k) qui reçoit en paramètres un réel x et un entier positif k , et qui retourne la valeur de l'expression suivante :

$$\frac{x^k}{k!}$$

1.5 pt Q4- Écrire la fonction **exponentielle** (x , n) qui reçoit en paramètres un réel x et un entier positif n , et qui retourne la valeur du développement limité de e^x , à l'ordre n .

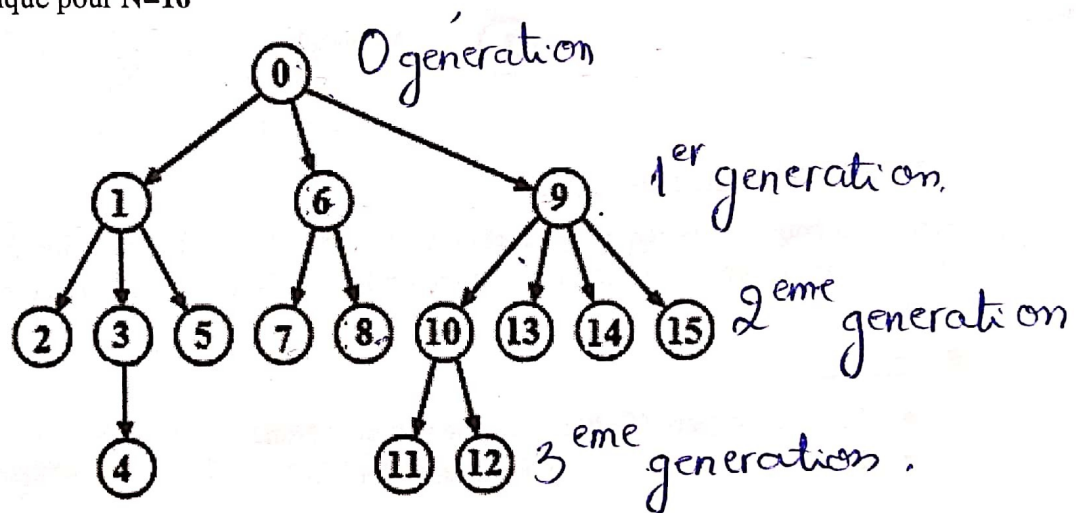
Partie III : Problème Python

Arbre généalogique

Le but de ce problème est de modéliser une structure élémentaire représentant un arbre généalogique « descendant » où on représente sur quelques générations la descendance issue d'un unique ancêtre commun, sans se préoccuper du sexe des différents membres de l'arbre. Ainsi, chaque membre de l'arbre est relié vers le haut à une unique personne qu'on appellera simplement son père, et vers le bas à tous ses enfants. À l'exception de l'ancêtre commun tout en haut de l'arbre qui n'a pas de père.

Si l'arbre généalogique est composé de N membres, alors chaque membre de l'arbre est représenté par un nombre entier positif unique compris entre 0 et $N-1$. Le plus grand ancêtre est représenté par le nombre 0.

Exemple : Arbre généalogique pour $N=16$



Représentation de l'arbre généalogique en Python :

En python, pour représenter un arbre généalogique composé de N membres, on utilise une liste G de longueur N , telle que pour tout entier $i \in \llbracket 0, N \rrbracket$:

- Si i est la racine de l'arbre, alors la valeur de l'élément $G[i]$ est `None`
- Sinon, la valeur de l'élément $G[i]$ est le père de l'élément i dans l'arbre.

Exemple :

L'arbre généalogique, dans l'exemple ci-dessus, sera représenté par la liste G de longueur 16 :

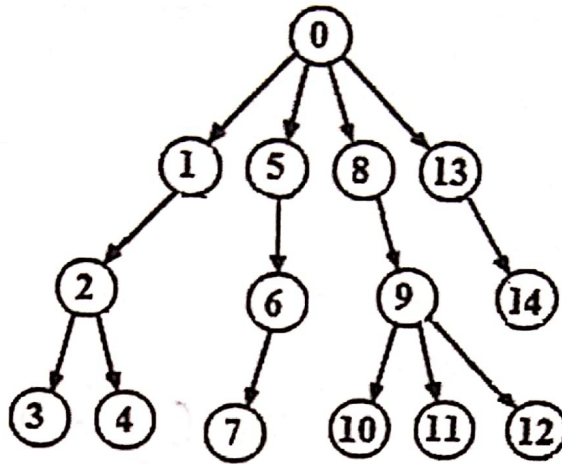
$G =$	[None,	0,	1,	1,	3,	1,	0,	6,	6,	0,	9,	10,	10,	9,	9,	9]
indices	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Les indices de la liste G représentent les membres de l'arbre généalogique :

- ✓ Le membre 0 est la racine de l'arbre, il n'a pas de père : $G[0] = \text{None}$.
- ✓ 0 est le père du membre 1 : $G[1] = 0$
- ✓ 6 est le père du membre 7 : $G[7] = 6$
- ✓ 10 est le père du membre 12 : $G[12] = 10$
- ✓ ...

NB : Pour plus de clarté, dans les exemples des fonctions de cette partie, on utilisera l'arbre généalogique précédent.

Q.1- Donner la liste qui représente l'arbre généalogique suivant :



Q.2- Père d'un membre de l'arbre

Écrire la fonction *pere* (*G*, *m*) qui reçoit en paramètres la liste *G* qui représente un arbre généalogique, et *m* un entier positif qui représente un membre de l'arbre *G*. La fonction retourne le père du membre *m* dans l'arbre *G*.

Exemples :

- La fonction *pere* (*G*, 0) retourne le nombre : None
- La fonction *pere* (*G*, 4) retourne le nombre : 3

Q.3- Liste des fils d'un membre

Écrire la fonction *liste_fils* (*G*, *p*) qui reçoit en paramètres la liste *G* qui représente un arbre généalogique, et *p* un entier positif qui représente un membre de l'arbre *G*. La fonction retourne la liste des fils du membre *p* dans l'arbre *G*.

Exemples :

- La fonction *liste_fils* (*G*, 8) retourne la liste : []
- La fonction *liste_fils* (*G*, 9) retourne la liste : [10, 13, 14, 15]

Q.4- Génération d'un membre

Dans un arbre généalogique, la **génération** de l'ancêtre commun est 0, la génération de ses enfants est 1, la génération des enfants de ses enfants est 2, ...

Écrire la fonction *generation* (*G*, *m*) qui reçoit en paramètres la liste *G* qui représente un arbre généalogique, et *m* un entier positif qui représente un membre de l'arbre *G*. La fonction retourne la génération de *m* dans l'arbre *G*.

Exemples :

- La fonction *generation* (*G*, 0) retourne le nombre 0
- La fonction *generation* (*G*, 4) retourne le nombre 3

Q.5- Hauteur de l'arbre généalogique

La hauteur de l'arbre généalogique est la plus grande génération des membres de cet arbre.

Écrire la fonction *hauteur* (*G*) qui reçoit en paramètres la liste *G* qui représente un arbre généalogique. La fonction retourne la hauteur de l'arbre *G*.

Exemple :

La fonction *hauteur* (*G*) retourne le nombre 4

Q.6- Descendant d'un membre

Soient *i* et *j* deux membres d'un arbre généalogique *G*. On dit que *i* est un descendant de *j* si *j* est le père de *i*, ou si *j* est le père du père de *i*, ou si *j* est le père du père du père de *i*, etc ...

Exemples :

- ✓ Le membre 12 est un descendant du membre 9
- ✓ Le membre 5 est un descendant du membre 1
- ✓ Le membre 2 n'est pas un descendant du membre 6

Écrire la fonction *descendant* (*G*, *i*, *j*) qui reçoit en paramètres la liste *G* qui représente un arbre généalogique, et *i* et *j* deux entiers positifs qui représentent deux membres de l'arbre *G*. La fonction retourne *True* si *i* est un descendant de *j*, sinon, la fonction retourne *False*.

Exemples :

- La fonction *descendant* (*G*, 14, 0) retourne *True*
- La fonction *descendant* (*G*, 5, 4) retourne *False*

Q.7- Plus proche ancêtre de deux membres

Soient *i* et *j* deux membres d'un arbre généalogique *G*. Le plus proche ancêtre de *i* et *j* est le membre de l'arbre qui est ancêtre commun à *i* et *j*, et ayant la plus grande génération des ancêtres communs à *i* et *j*. Le plus proche ancêtre existe toujours, et il est même unique, puisque tout membre de l'arbre est ancêtre de lui-même.

Exemples :

- ✓ Le membre 1 est le plus proche ancêtre des deux membres 2 et 4
- ✓ Le membre 0 est le plus proche ancêtre des deux membres 7 et 12

Écrire la fonction *plus_proche_ancetre* (*G*, *i*, *j*) qui reçoit en paramètres la liste *G* qui représente un arbre généalogique, et *i* et *j* deux entiers positifs qui représentent deux membres de l'arbre *G*. La fonction retourne le plus proche ancêtre de *i* et *j* dans l'arbre *G*.

Exemples :

- La fonction *plus_proche_ancetre* (*G*, 5, 11) retourne : 0
- La fonction *plus_proche_ancetre* (*G*, 12, 15) retourne : 9
- La fonction *plus_proche_ancetre* (*G*, 6, 7) retourne : 6

Q.8- Distance entre deux membres

On considère deux membres i et j de l'arbre généalogique G . La **distance** entre i et j est la longueur du plus court chemin menant de i à j dans l'arbre. Ce chemin est celui qui part de i , et qui remonte jusqu'au plus proche ancêtre de i et j , puis qui redescend jusqu'à j .

Exemples :

- ✓ La distance entre les membres 4 et 11 est : 6
- ✓ La distance entre les membres 7 et 8 est : 2
- ✓ La distance entre les membres 12 et 15 est : 3

Écrire la fonction *distance* (G, i, j) qui reçoit en paramètres la liste G qui représente un arbre généalogique, et i et j deux entiers positifs qui représentent deux membres de l'arbre G . La fonction retourne la distance entre i et j dans l'arbre G .

Exemples :

- La fonction *distance* ($G, 8, 4$) retourne : 5
- La fonction *distance* ($G, 5, 2$) retourne : 2

Q.9- Plus court chemin entre deux membres

On considère deux membres i et j de l'arbre généalogique G . Le **plus court chemin** de i à j est le chemin partant de i et remontant jusqu'au plus proche ancêtre commun de i et j , puis redescendant jusqu'à j .

Exemples :

- ✓ Le plus court chemin du membre 4 au membre 11 est : '4-3-1-0-9-10-11'
- ✓ Le plus court chemin du membre 8 au membre 7 est : '8-6-7'
- ✓ Le plus court chemin du membre 12 au membre 15 est : '12-10-9-15'

Écrire la fonction *plus_court_chemin* (G, i, j) qui reçoit en paramètres la liste G qui représente un arbre généalogique, et i et j deux entiers positifs qui représentent deux membres de l'arbre G . La fonction retourne la chaîne de caractères qui contient le chemin le plus court du membre i au membre j dans l'arbre G .

Exemples :

- La fonction *plus_court_chemin* ($G, 5, 9$) retourne : '5-1-0-9'
- La fonction *plus_court_chemin* ($G, 8, 4$) retourne : '8-6-0-1-3-4'

~~~~~ FIN DE L'ÉPREUVE ~~~~~



Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

**Remarques générales :**

- Cette épreuve est composée d'un exercice et de trois parties toutes indépendantes ;
- Toutes les instructions et les fonctions demandées seront écrites en Python ;
- Les questions non traitées peuvent être admises pour aborder les questions ultérieures ;
- Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

~~~~~

Exercice : (4 points)

Les coefficients binomiaux

Un **coefficient binomial** est défini pour deux entiers positifs **n** et **k** tels que $k \leq n$. C'est le nombre de parties de **k** éléments dans un ensemble de **n** éléments. On le note : $\binom{n}{k}$, et sa valeur est calculée par la formule suivante :

$$\binom{n}{k} = \frac{n!}{k! (n-k)!}$$

1 pt Q1- Écrire la fonction **fact(p)** qui reçoit en paramètre un entier positif **p**, et qui retourne la valeur de **factorielle p** : $p! = 1 * 2 * 3 * \dots * (p-1) * p$.

NB : La fonction **fact(0)** retourne 1

1 pt Q2- Écrire la fonction **produit(n, k)** qui reçoit en paramètres deux entiers positifs **n** et **k** tels que $n \geq k$, et qui retourne la valeur du produit : $n * (n-1) * (n-2) * \dots * (n - (k-1))$

0.75 pt Q3- Écrire la fonction **binomial(n, k)** qui reçoit en paramètres deux entiers positifs **n** et **k** tels que $n \geq k$, et qui retourne la valeur du coefficient binomial $\binom{n}{k}$.

Exemple : La fonction **binomial(6, 3)** retourne le nombre **20**

1.25 pt Q4- Écrire la fonction **liste_binomiaux(n)** qui reçoit en paramètre un entier positif **n**, et qui retourne la liste des coefficients binomiaux $\binom{n}{k}$ tel que : $k = 0, 1, 2, 3, \dots, n$

Exemple : La fonction **liste_binomiaux(6)** retourne la liste **[1, 6, 15, 20, 15, 6, 1]**

Partie III : Problème*Carré magique*

On considère un entier n strictement positif. Un **carré magique** d'ordre n est une matrice carrée d'ordre n (n lignes et n colonnes), qui contient des nombres entiers strictement positifs. Ces nombres sont disposés de sorte que les sommes sur chaque ligne, les sommes sur chaque colonne et les sommes sur chaque diagonale principale soient égales. La valeur de ces sommes est appelée : **constante magique**.

Exemple :

Carré magique d'ordre 3, sa constante magique 45

21	7	17	→ 45
11	15	19	→ 45
13	23	9	→ 45
↙ 45	↓ 45	↓ 45	↘ 45

Représentation d'une matrice carrée en Python :

Pour représenter une matrice carrée d'ordre n (n lignes et n colonnes), on utilise une liste qui contient n listes, toutes de même longueur n .

Exemple :

4	7	10	3
3	2	9	6
13	0	5	8
7	1	6	25

Cette matrice carrée d'ordre 4 est représentée par la liste **M**, composée de 4 listes de taille 4 chacune :

M = [[4, 7, 10, 3], [3, 2, 9, 6], [13, 0, 5, 8], [7, 1, 6, 25]]

M[i] est la liste qui représente la ligne d'indice i dans **M**.

Exemples :

- **M[0]** est la liste [4, 7, 10, 3]
- **M[2]** est la liste [13, 0, 5, 8]

M[i][j] est l'élément à la $i^{\text{ème}}$ ligne et la $j^{\text{ème}}$ colonne, dans **M**

Exemples :

- **M[0][1]** est l'élément 7
- **M[2][1]** est l'élément 0

I.- Opérations sur une matrice carrée

Q.1- Écrire la fonction `somme_ligne (M, i)`, qui reçoit en paramètres une matrice carrée **M** contenant des nombres, et un entier **i** qui représente l'indice d'une ligne dans **M**. La fonction retourne la somme des nombres de la ligne d'indice **i** dans **M**.

Exemple :

La fonction `somme_ligne (M, 1)` retourne la somme $3+2+9+6 = 20$

Q.2- Écrire la fonction `somme_colonne (M, j)`, qui reçoit en paramètres une matrice carrée **M** contenant des nombres, et un entier **j** qui représente l'indice d'une colonne dans **M**. La fonction retourne la somme des éléments de la colonne d'indice **j** dans **M**.

Exemple :

La fonction `somme_colonne (M, 0)` retourne la somme $4+3+13+7 = 27$

Q.3- Écrire la fonction `somme_diag1 (M)`, qui reçoit en paramètre une matrice carrée **M** contenant des nombres, et qui retourne la somme des éléments de la première diagonale principale dans **M**.

Exemple :

La fonction `somme_diag1 (M)` retourne la somme $4+2+5+25 = 36$

Q.4- Écrire la fonction `somme_diag2 (M)`, qui reçoit en paramètre une matrice carrée **M** contenant des nombres, et qui retourne la somme des éléments de la deuxième diagonale principale dans **M**. (La deuxième diagonale principale part du coin en haut à droite, jusqu'au coin en bas à gauche)

Exemple :

La fonction `somme_diag2 (M)` retourne la somme $3+9+0+7 = 19$

II- Carré magique

Q.5- Écrire la fonction `carre_magique (C)`, qui reçoit en paramètre une matrice carrée **C** contenant des entiers strictement positifs, et qui retourne :

True, si la matrice **C** est un carré magique : les sommes sur chaque ligne, sur chaque colonne et sur chaque diagonale principale sont toutes égales

False, sinon.

Exemples :

$A =$

21	7	17
11	15	19
13	23	9

$B =$

7	1	6
1	15	9
3	2	4

- La fonction `carre_magique (A)` retourne **True**
- La fonction `carre_magique (B)` retourne **False**

III- Carré magique normal

Un **carré magique normal** d'ordre n est un carré magique d'ordre n , constitué de tous les nombres entiers positifs compris entre 1 et n^2 .

Exemple :

Carrée magique normal d'ordre 4 , composé des nombres entiers : $1, 2, 3, \dots, 15, 16$.

4	14	15	1
9	7	6	12
5	11	10	8
16	2	3	13

NB : Il n'existe pas de carré magique normal d'ordre 2 .

Q.6.a- Écrire la fonction **magique_normal(C)**, qui reçoit en paramètre une matrice carrée **C** qui représente un carré magique. La fonction retourne **True** si le carré magique **C** est normal, sinon, elle retourne **False**.

Exemples:

- La fonction **magique_normal** ([[8, 1, 6], [3, 5, 7], [4, 9, 2]]) retourne **True**
- La fonction **magique_normal** ([[21, 7, 17], [11, 15, 19], [13, 23, 9]]) retourne **False**

Q.6.b- Déterminer la complexité de la fonction **magique_normal(C)**.

IV- Construction d'un carré magique normal d'ordre impair

La méthode **siamoise** est une méthode qui permet de construire un carré magique normal d'ordre n impair.

Le principe de cette méthode est le suivant :

1. Créer une matrice carrée d'ordre n , remplie de 0 .
2. Placer le nombre 1 au milieu de la ligne d'indice 0 .
3. Décaler d'une case vers la droite puis d'une case vers le haut pour placer le nombre 2 , et faire de même pour le nombre 3 , puis le nombre 4 , ... jusqu'au nombre n^2 .

Le déplacement doit respecter les deux règles suivantes (voir l'exemple dans la page suivante) :

Si la pointe de la flèche sort du carré, revenir de l'autre côté, comme si le carré était enroulé sur un tore.

Si la prochaine case est occupée par un entier non nul, alors il faut décaler d'une case vers le bas.

Exemple :

Construction d'un carré magique normal d'ordre 5

17	24	1	8	15
23	5	7	14	16
4	6	13	20	22
10	12	19	21	3
11	18	25	2	9

Q.7- Écrire la fonction **matrice_nulle(n)**, qui reçoit en paramètre un entier **n** strictement positif, et qui retourne une liste qui représente la matrice carrée d'ordre **n**, remplie de 0.

Exemple :

La fonction **matrice_nulle(5)** retourne la matrice suivante :

[[0,0,0,0,0], [0,0,0,0,0], [0,0,0,0,0], [0,0,0,0,0] , [0,0,0,0,0]]

Q.8- Écrire la fonction **siamoise(n)**, qui reçoit en paramètre un entier positif **n** impair. En utilisant le principe de la méthode siamoise, la fonction retourne la matrice carrée qui représente le carré magique normal d'ordre **n**.

Exemple :

La fonction **siamoise(7)** retourne la matrice carrée qui représente le carré magique normale d'ordre 7 suivant :

30	39	48	1	10	19	28
38	47	7	9	18	27	29
46	6	8	17	26	35	37
5	14	16	25	34	36	45
13	15	24	33	42	44	4
21	23	32	41	43	3	12
22	31	40	49	2	11	20

Q.9- Écrire la fonction, de complexité constante, **constante_magique(n)**, qui reçoit en paramètre un entier positif **n** impair, et qui retourne la valeur de la constante magique du carré magique normal d'ordre **n**.

~~~~~ FIN DE L'ÉPREUVE ~~~~~

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

**Remarques générales :**

- Cette épreuve est composée d'un exercice et de trois parties tous indépendants ;
- Toutes les instructions et les fonctions demandées seront écrites en Python ;
- Les questions non traitées peuvent être admises pour aborder les questions ultérieures ;
- Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

~~~~~

Exercice : (4 points)

Les coefficients binomiaux

- 1 pt Q1-** Écrire la fonction **fact (p)** qui reçoit en paramètre un entier positif **p**, et qui retourne la valeur de **factorielle p** : **p! = 1 * 2 * 3 * ... * (p-1) * p**.

NB : La fonction **fact(0)** retourne 1

- 0.5 pt Q2-** Déterminer la complexité de la fonction **fact (p)**

Un **coefficient binomial** est défini pour deux entiers positifs **n** et **k** tels que **k ≤ n**. C'est le nombre de parties de **k** éléments dans un ensemble de **n** éléments. On le note : $\binom{n}{k}$, et sa valeur est calculée par la formule suivante :

$$\binom{n}{k} = \frac{n!}{k! * (n - k)!}$$

- 1 pt Q3-** Écrire la fonction **binomial (n, k)** qui reçoit en paramètres deux entiers positifs **n** et **k** tels que **k ≤ n**, et qui retourne la valeur du coefficient binomial $\binom{n}{k}$.

- 1.5 pt Q4-** Écrire la fonction **binomiaux (n)**, qui reçoit en paramètre un entier positif **n**, et qui affiche tous les coefficients binomiaux $\binom{n}{k}$ tel que : **k = 0, 1, 2, ..., n**.

Exemple :

La fonction **binomiaux (6)** affiche les nombres : **1 , 6 , 15 , 20 , 15 , 6 et 1**

Partie III : Problème

Transformée de 'Burrows-Wheeler'

La **transformée de Burrows-Wheeler**, couramment désignée par l'acronyme **BWT** (pour *Burrows-Wheeler Transform*) est une technique inventée par Michael Burrows et David Wheeler, et elle a été publiée en 1994. Cette technique permet de réorganiser les données : c'est une transformation qui, à partir d'un texte donné, produit un autre texte contenant exactement les mêmes caractères, mais dans un autre ordre, pour lequel les répétitions de caractères ont tendance à être contiguës.

Le but de cette partie est de réaliser l'algorithme de la transformée de Burrows-Wheeler.

1- Rotation d'une chaîne de caractères

On considère une chaîne de caractères **T** non vide, et un entier **p** tel que : $0 \leq p < \text{taille de } T$.

La **rotation de T d'indice p** est la chaîne de caractères obtenue par la concaténation des deux sous-chaînes suivantes :

La sous-chaîne de T, composée de la suite des caractères en partant de la position p, jusqu'à la dernière position dans T ;

et de la sous-chaîne de T, composée de la suite des caractères en partant de la première position de T, jusqu'à la position p-1 dans T.

Exemples :

T = 'daacdacdab'

La rotation de **T** d'indice **0** est la chaîne : **'daacdacdab'** + " = **'daacdacdab'**

La rotation de **T** d'indice **1** est la chaîne : **'aacdacdab'** + **'d'** = **'aacdacdabd'**

La rotation de **T** d'indice **2** est la chaîne : **'acdacdab'** + **'da'** = **'acdacdabda'**

La rotation de **T** d'indice **9** est la chaîne : **'b'** + **'daacdacda'** = **'bdaacdacda'**

Q.1- Écrire la fonction **rotation(T, p)**, qui reçoit en paramètres une chaîne de caractères **T** non vide, et un entier **p** tel $0 \leq p < \text{taille de } T$. La fonction retourne la rotation de **T** d'indice **p**.

2- Liste des rotations d'une chaîne de caractères

Q.2- Écrire la fonction **liste_rotations(T)**, qui reçoit en paramètre une chaîne de caractères **T** non vide, et qui retourne une liste **R** qui contient toutes les rotations possibles de **T**.

Exemple :

T = 'daacdacdab' (la taille de la chaîne **T** est **10**)

La fonction **liste_rotations(T)** retourne la liste des **10** rotations possibles :

R = [**'daacdacdab'**, **'aacdacdabd'**, **'acdacdabda'**, **'cdacdabdaa'**, **'dacdabdaac'**, **'acdabdaacd'**, **'cdabdaacda'**, **'dabdaacdac'**, **'abdaacdacd'**, **'bdaacdacda'**]

3- Tri de la liste des rotations

Q.3- Écrire la fonction **tri_rotations (R)**, qui reçoit en paramètre la liste **R** qui contient toutes les rotations d'une chaîne de caractères. La fonction trie les éléments de **R** dans l'ordre alphabétique.

Exemple :

R = ['daacdacadab', 'aacdacadabd', 'acdacadabda', 'cdacadabaa', 'dacadabdaac', 'acdabdaacd', 'cdabdaacda', 'dabdaacdac', 'abdaacdacd', 'bdaacdacda']

Après l'appel de la fonction **tri_rotations (R)**, on obtient la liste triée :

['aacdacadabd', 'abdaacdacd', 'acdabdaacd', 'acdacadabda', 'bdaacdacda', 'cdabdaacda', 'cdacadabaa', 'daacdacadab', 'dabdaacdac', 'dacadabdaac']

4- Position de la chaîne initiale dans la liste des rotations triée

La liste **R** contient toutes les rotations de la chaîne de caractères **T**. On suppose que la liste **R** est triée dans l'ordre alphabétique. La chaîne **T** est un élément de la liste **R** (**T** est la rotation d'indice **0**). Et on veut chercher la position de **T** dans la liste **R** triée. Pour cela, on propose d'utiliser le principe de la **recherche dichotomique**.

La **recherche dichotomique** est un algorithme de complexité logarithmique, qui permet de rechercher la position d'un élément **x** dans une liste **L** triée. Le principe de cet algorithme est le suivant :

1. Calculer la position **m** du milieu de la liste **L** ;
2. Si **L[m] = x**, alors retourner **m** (**m** est la position de **x** dans **L**) ;
3. Si **x < L[m]**, alors reprendre l'étape (1) en considérant la première moitié de la liste **L** ;
4. Si **x > L[m]**, alors reprendre l'étape (1) en considérant la deuxième moitié de la liste **L**.

Q.4.a- Écrire la fonction **position(T, R)**, qui reçoit en paramètres une chaîne de caractères **T**, non vide, et la liste **R** de toutes les rotations de **T**, triée dans l'ordre alphabétique. La fonction retourne la position de la chaîne **T** dans la liste triée **R**, en utilisant le principe de la recherche dichotomique.

Exemple :

T = 'daacdacadab'

La liste des rotations de **T**, triée dans l'ordre alphabétique est :

R = ['aacdacadabd', 'abdaacdacd', 'acdabdaacd', 'acdacadabda', 'bdaacdacda', 'cdabdaacda', 'cdacadabaa', '**daacdacadab**', 'dabdaacdac', 'dacadabdaac']

La fonction **position (T, R)** retourne le nombre **7**, qui représente l'indice de la chaîne **T** dans la liste des rotations triée **R**.

Q.4.b- Déterminer la complexité de la fonction **position(T, R)**.

5- Transformée de Burrows-Wheeler

La transformée de Burrows-Wheeler se contente de réorganiser les caractères d'une chaîne de caractères **T**, de manière à obtenir un autre texte contenant exactement les mêmes caractères de **T**, mais dans un autre ordre, pour lequel les répétitions de caractères ont tendance à être contiguës.

Le principe de la transformée de Burrows-Wheeler est le suivant :

1. On construit la liste **R** de toutes les rotations possibles de la chaîne **T** ;
2. On trie les éléments de la liste **R** dans l'ordre alphabétique ;
3. On recherche la position **p** de la chaîne **T** dans la liste **R** triée ;
4. On construit la chaîne **B** composée des derniers caractères des chaînes de la liste **R** triée ;
5. Le résultat de la transformée de Burrows-Wheeler est le tuple **(B, p)**.

Example :

T = 'daacdac dab'

La liste des rotations de **T**, triée dans l'ordre alphabétique est :

```
R = [ 'aacdacdabd', 'abdaacdacd', 'acdabdaacd', 'acdacdabda', 'bdaacdacda', 'cdabdaacda',  
'cdacdabdaa', 'daacdacdab', 'dabdaacdac', 'dacdabdaac' ]
```

La position de **T** dans **R** est : **p=7**.

La chaîne composée des derniers caractères des chaînes de la liste **R** triée est : **B = 'dddaaaabcc'**.

Le résultat de la transformée de Burrows-Wheeler est le tuple : ('dddaaaabcc' , 7)

Q.5- Écrire la fonction **BWT (T)**, qui reçoit en paramètre une chaîne de caractères **T** non vide, et qui retourne le tuple **(B, p)**.

Example :

T = 'daacdacdad'

La fonction **BWT (T)** retourne le tuple : ('dddaaaabcc' , 7)

6- La compression RLE

La transformée de Burrows-Wheeler est à la base de certains algorithmes de compression. Elle est utilisée avant la compression de données.

La **compression RLE (Run Length Encoding)** est un algorithme qui permet de compresser un texte contenant des suites de caractères identiques. Il consiste à repérer la répétition des caractères dans le texte, et de les remplacer par un chiffre qui représente le nombre de répétition d'un caractère, suivi du caractère répété.

Par exemple, on considère la chaîne **S = 'BBBBZZZZZZZZZZZZZZZZZZZZZZZZZZZZBBBX'**

Après la compression de la chaîne **S**, on obtient la chaîne **'4B9Z9Z5Z3B1X'**

Q.6.a- Écrire la fonction **decompression(ch)**, qui reçoit en paramètre une chaîne de caractères **ch** qui représente une chaîne compressée par l'algorithme de compression RLE. La fonction retourne la chaîne décompressée.

Example :

La fonction **decompression** ('4B9Z9Z5Z3B1X') retourne la chaîne de caractères décompressée :

'BBBBZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZBBBX'

Q.6.b- Écrire la fonction **RLE (S)**, qui reçoit en paramètre une chaîne de caractères **S**. La fonction retourne la chaîne compressée de **S**, en utilisant le principe de la compression **RLE**.

Exemple :

S = 'BBBBZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZBBBX'

La fonction **RLE (S)** retourne la chaîne de caractères **'4B9Z9Z5Z3B1X'**

NB : Dans cet exemple, le caractère '**Z**' est répété **23** fois. La compression de cette suite de caractères donne pour résultat : '**9Z9Z5Z**', et non pas '**23Z**'.

≈≈≈≈≈ **FIN DE L'ÉPREUVE** ≈≈≈≈≈

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Remarques générales :

- Cette épreuve est composée d'un exercice et de trois parties tous indépendants ;
- Toutes les instructions et les fonctions demandées seront écrites en Python ;
- Les questions non traitées peuvent être admises pour aborder les questions ultérieures ;
- Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

~~~~~

**Exercice : (4 points)**

*Somme de nombres entiers*

- 1 pt Q.1-** Écrire la fonction **somme (n)** qui reçoit en paramètre un entier strictement positif **n**, et qui retourne la valeur de la somme :  $1 + 2 + 3 + \dots + n$

**Exemple :** La fonction **somme(4)** retourne 10

- 0.5 pt Q.2-** Déterminer la complexité de la fonction **somme(n)**.

- 0.75 pt Q.3-** Écrire la fonction **terme (k)** qui reçoit en paramètre un entier **k** strictement positif, et qui retourne la valeur de l'expression suivante :

$$\frac{1}{1 + 2 + 3 + \dots + k}$$

**Exemple :** La fonction **terme(4)** retourne le nombre 0.4

- 0.5 pt Q.4-** Déterminer la complexité de la fonction **terme(k)**.

- 1.25 pt Q.5-** Écrire la fonction **sigma (n)** qui reçoit en paramètre un entier strictement positif **n**, et qui retourne la valeur de la somme suivante :

$$1 + \frac{1}{1 + 2 + 3 + \dots + k} + \frac{1}{1 + 2 + 3 + \dots + k} + \dots + \frac{1}{1 + 2 + 3 + \dots + k}$$

## Partie III : Problème

### *Transformée de 'Burrows-Wheeler'*

La **transformée de Burrows-Wheeler**, couramment désignée par l'acronyme **BWT** (pour *Burrows-Wheeler Transform*) est une technique inventée par Michael Burrows et David Wheeler, et elle a été publiée en 1994. Cette technique permet de réorganiser les données : c'est une transformation qui, à partir d'un texte donné, produit un autre texte contenant exactement les mêmes caractères, mais dans un autre ordre, pour lequel les répétitions de caractères ont tendance à être contiguës.

Le but de cette partie est de réaliser l'algorithme de la transformée de Burrows-Wheeler.

#### 1- Rotation d'une chaîne de caractères

On considère une chaîne de caractères **T** non vide, et un entier **p** tel que :  $0 \leq p < \text{taille de } T$ .

La **rotation de T d'indice p** est la chaîne de caractères obtenue par la concaténation des deux sous-chaînes suivantes :

*La sous-chaîne de T, composée de la suite des caractères en partant de la position **p**, jusqu'à la dernière position dans T ;*

*et de la sous-chaîne de T, composée de la suite des caractères en partant de la première position de T, jusqu'à la position **p-1** dans T.*

#### Exemples :

**T = 'daacdacdab'**

La rotation de **T** d'indice **0** est la chaîne : **'daacdacdab'** + " = **'daacdacdab'**

La rotation de **T** d'indice **1** est la chaîne : **'aacdacdab'** + **'d'** = **'aacdacdabd'**

La rotation de **T** d'indice **2** est la chaîne : **'acdacdab'** + **'da'** = **'acdacdabda'**

La rotation de **T** d'indice **9** est la chaîne : **'b'** + **'daacdacda'** = **'bdaacdacda'**

**Q.1-** Écrire la fonction **rotation(T, p)**, qui reçoit en paramètres une chaîne de caractères **T** non vide, et un entier **p** tel  $0 \leq p < \text{taille de } T$ . La fonction retourne la rotation de **T** d'indice **p**.

#### 2- Liste des rotations d'une chaîne de caractères

**Q.2-** Écrire la fonction **liste\_rotations(T)**, qui reçoit en paramètre une chaîne de caractères **T** non vide, et qui retourne une liste **R** qui contient toutes les rotations possibles de **T**.

#### Exemple :

**T = 'daacdacdab'** (la taille de la chaîne **T** est **10**)

La fonction **liste\_rotations(T)** retourne la liste des **10** rotations possibles :

**R** = [ **'daacdacdab'**, **'aacdacdabd'**, **'acdacdabda'**, **'cdacdabdaa'**, **'dacdabdaac'**, **'acdabdaacd'**, **'cdabdaacda'**, **'dabdaacdac'**, **'abdaacdacd'**, **'bdaacdacda'** ]

### 3- Tri de la liste des rotations

**Q.3-** Écrire la fonction **tri\_rotations (R)**, qui reçoit en paramètre la liste **R** qui contient toutes les rotations d'une chaîne de caractères. La fonction trie les éléments de **R** dans l'ordre alphabétique.

Exemple :

**R** = [ 'daacdacadab', 'aacdacadabd', 'acdacadabda', 'cdacadabdaa', 'dacdabdaac', 'acdabdaacd', 'cdabdaacda', 'dabdaacdac', 'abdaacdacd', 'bdaacdacda' ]

Après l'appel de la fonction **tri\_rotations (R)**, on obtient la liste triée :

[ 'aacdacadabd', 'abdaacdacd', 'acdabdaacd', 'acdacadabda', 'bdaacdacda', 'cdabdaacda', 'cdacadabdaa', 'daacdacadab', 'dabdaacdac', 'dacdabdaac' ]

### 4- Position de la chaîne initiale dans la liste des rotations triée

La liste **R** contient toutes les rotations de la chaîne de caractères **T**. On suppose que la liste **R** est triée dans l'ordre alphabétique. La chaîne **T** est un élément de la liste **R** (**T** est la rotation d'indice **0**). Et on veut chercher la position de **T** dans la liste **R** triée. Pour cela, on propose d'utiliser le principe de la **recherche dichotomique**.

La **recherche dichotomique** est un algorithme de complexité logarithmique, qui permet de rechercher la position d'un élément **x** dans une liste **L** triée. Le principe de cet algorithme est le suivant :

1. Calculer la position **m** du milieu de la liste **L** ;
2. Si **L[m] = x**, alors retourner **m** (**m** est la position de **x** dans **L**) ;
3. Si **x < L[m]**, alors reprendre l'étape (1) en considérant la première moitié de la liste **L** ;
4. Si **x > L[m]**, alors reprendre l'étape (1) en considérant la deuxième moitié de la liste **L**.

**Q.4-** Écrire la fonction **position (T, R)**, qui reçoit en paramètres une chaîne de caractères **T**, non vide, et la liste **R** de toutes les rotations de **T**, triée dans l'ordre alphabétique. La fonction retourne la position de la chaîne **T** dans la liste triée **R**, en utilisant le principe de la recherche dichotomique.

Exemple :

**T** = 'daacdacadab'

La liste des rotations de **T**, triée dans l'ordre alphabétique est :

**R** = [ 'aacdacadabd', 'abdaacdacd', 'acdabdaacd', 'acdacadabda', 'bdaacdacda', 'cdabdaacda', 'cdacadabdaa', '**daacdacadab**', 'dabdaacdac', 'dacdabdaac' ]

La fonction **position (T, R)** retourne le nombre **7**, qui représente l'indice de la chaîne **T** dans la liste des rotations triée **R**.

### 5- Transformée de Burrows-Wheeler

La transformée de Burrows-Wheeler se contente de réorganiser les caractères d'une chaîne de caractères **T**, de manière à obtenir un autre texte contenant exactement les mêmes caractères de **T**, mais dans un autre ordre, pour lequel les répétitions de caractères ont tendance à être contiguës.

Le principe de la transformée de Burrows-Wheeler est le suivant :

1. On construit la liste **R** de toutes les rotations possibles de la chaîne **T** ;
2. On trie les éléments de la liste **R** dans l'ordre alphabétique ;
3. On recherche la position **p** de la chaîne **T** dans la liste **R** triée ;
4. On construit la chaîne **B** composée des derniers caractères des chaînes de la liste **R** triée ;
5. Le résultat de la transformée de Burrows-Wheeler est le tuple **(B, p)**.

**Exemple :**

**T = 'daacdacdab'**

La liste des rotations de **T**, triée dans l'ordre alphabétique est :

**R** = [ 'aacdacdabd', 'abdaacdacd', 'acdabdaacd', 'acdacdabda', 'bdaacdacda', 'cdabdaacda', 'cdacdabdaa', 'daacdacdab', 'dabdaacdac', 'dacdabdaac' ]

La position de **T** dans **R** est : **p=7**.

La chaîne composée des derniers caractères des éléments de la liste **R** triée est : **B = 'dddaaaabcc'**.

Le résultat de la transformée de Burrows-Wheeler est le tuple : **( 'dddaaaabcc' , 7 )**

**Q.5-** Écrire la fonction **BWT (T)**, qui reçoit en paramètre une chaîne de caractères **T** non vide, et qui retourne le tuple **(B, p)**.

**Exemple :**

**T = 'daacdacdab'**

La fonction **BWT (T)** retourne le tuple : **( 'dddaaaabcc' , 7 )**

~~~~~ **FIN DE L'ÉPREUVE** ~~~~~


Épreuve d'Informatique – Session 2019 – Filière MP

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Remarques générales :

- Cette épreuve est composée d'un exercice et de deux parties tous indépendants ;
- Toutes les instructions et les fonctions demandées seront écrites en Python ;
- Les questions non traitées peuvent être admises pour aborder les questions ultérieures ;
- Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

~~~~~

**Exercice : (4 points)*****Médian d'une liste de nombres***

**1 pt**    **Q 1-** Écrire la fonction **grands (L, x)** qui reçoit en paramètres une liste de nombres **L**, et un élément **x** de **L**. La fonction renvoie le nombre d'éléments de **L** qui sont supérieurs strictement à **x**.

**1 pt**    **Q 2-** Déterminer la complexité de la fonction **grands (L, x)**, et justifier votre réponse.

**0.5 pt**    **Q 3-** Écrire la fonction **petits (L, x)** qui reçoit en paramètres une liste de nombres **L**, et un élément **x** de **L**. La fonction renvoie le nombre d'éléments de **L** qui sont inférieurs strictement à **x**.

**L** est une liste de taille **n** qui contient des nombres, et **m** un élément de **L**.

L'élément **m** est un **médian** de **L**, si les deux conditions suivantes sont vérifiées :

- Le nombre d'éléments de **L**, qui sont supérieurs strictement à **m**, est inférieur ou égale à **n/2**
- Le nombre d'éléments de **L**, qui sont inférieurs strictement à **m**, est inférieur ou égale à **n/2**

**Exemple :** On considère la liste **L = [ 25 , 12 , 6 , 17, 3 , 10 , 20 , 12 , 15 , 38 ]**, de taille **n=10**.

L'élément **12** est un médian de **L**, car :

- **3** éléments de **L** sont supérieurs strictement à **12**, et **3 ≤ n/2** ;
- **5** éléments de **L** sont inférieurs strictement à **12**, et **5 ≤ n/2**.

**1 pt**    **Q 4-** Écrire la fonction **median (L)** qui reçoit en paramètre une liste de nombres **L** non vide, et qui renvoie un élément médian de la liste **L**.

**0.5 pt**    **Q 5-** Déterminer la complexité de la fonction **median(L)**, et justifier votre réponse.

**Partie II :***Grille binaire*

Une **grille binaire** de **n** lignes et **p** colonnes, est une grille dans laquelle chaque case est de **couleur blanche**, ou bien de **couleur noire**.

**Exemple :**

	0	1	2	3	4	5	6	7	8	9	10	11
0												
1												
2												
3												
4												
5												
6												
7												
8												
9												

**Figure 1 :** Exemple de grille binaire de **10** lignes et **12** colonnes

**Représentation de la grille binaire :**

Pour représenter une grille binaire de **n** lignes et **p** colonnes, on utilise une matrice **G** de **n** lignes et **p** colonnes.

Chaque case de la grille **G** est représentée par un tuple **(i, j)** avec  $0 \leq i < n$  et  $0 \leq j < p$ , tels que :

$G[i, j] = 1$ , si la couleur de la case **(i, j)** est **blanche** ;

$G[i, j] = 0$ , si la couleur de la case **(i, j)** est **noire**.

**Exemple :**

La grille binaire de la **figure 1** est représentée par la matrice **G** de **10** lignes et **12** colonnes, suivante :

1	1	1	1	0	0	1	1	1	1	1	1	1
1	1	1	0	0	1	1	0	1	1	1	1	1
1	1	0	0	1	1	0	0	1	1	1	1	1
1	1	1	0	0	0	0	1	1	1	1	1	1
1	0	1	1	1	0	1	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	0	0	0	0
1	1	0	1	1	0	1	0	0	0	1	0	0
1	0	1	0	0	1	1	0	0	1	1	1	1
1	0	0	0	1	1	1	0	1	1	1	1	1
1	1	1	0	0	1	1	1	1	1	1	1	1

## II. 2- Représentation de la grille binaire par une liste de listes

Dans la suite de cette partie, pour représenter une grille binaire de  $n$  lignes et  $p$  colonnes, on utilise une liste  $G$  contenant  $n$  listes toutes de même longueur  $p$ .

Chaque case de la grille  $G$  est représentée par un tuple  $(i, j)$  avec  $0 \leq i < n$  et  $0 \leq j < p$ , tels que :

$G[i][j]=1$ , si la couleur de la case  $(i, j)$  est **blanche** ;

$G[i][j]=0$ , si la couleur de la case  $(i, j)$  est **noire**.

Pour plus de clarté, tous les exemples qui suivront, seront appliqués sur la grille binaire de la **figure 1**.

### Exemple :

La grille binaire de la **figure 1** est représentée par la liste  $G$  suivante, composée de **10** listes, de taille **12** chacune :

$G = [$   $[1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1]$  ,  $[1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1]$  ,  
 $[1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1]$  ,  $[1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1]$  ,  
 $[1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1]$  ,  $[1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0]$  ,  
 $[1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0]$  ,  $[1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1]$  ,  
 $[1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1]$  ,  $[1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1]$   
 $]$

i/j	0	1	2	3	4	5	6	7	8	9	10	11
0	1	1	1	1	0	0	1	1	1	1	1	1
1	1	1	1	0	0	1	1	0	1	1	1	1
2	1	1	0	0	1	1	0	0	1	1	1	1
3	1	1	1	0	0	0	0	1	1	1	1	1
4	1	0	1	1	1	0	1	1	1	1	1	1
5	1	0	0	1	1	1	0	1	1	0	0	0
6	1	1	0	1	1	1	0	0	0	0	1	0
7	1	0	1	0	0	1	1	0	0	1	1	1
8	1	0	0	0	1	1	1	0	1	1	1	1
9	1	1	1	0	0	1	1	1	1	1	1	1

$G[i][j]$  est l'élément qui se trouve à la  $i^{\text{ème}}$  ligne et la  $j^{\text{ème}}$  colonne dans  $G$ .

Exemples :

- $G[0][3]$  est l'élément 1
- $G[1][4]$  est l'élément 0.

$G[i]$  est la ligne d'indice  $i$  dans  $G$ .

Exemples :

- $G[2]$  est la liste  $[1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1]$ , qui représente la ligne d'indice 2 dans  $G$ .
- $G[5]$  est la liste  $[1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0]$ , qui représente la ligne d'indice 5 dans  $G$ .

Q 2- À partir de la liste  $G$ , de l'exemple ci-dessus, donner le résultat de chacune des expressions suivantes :

$G[4][2]$  ,  $\text{len}(G[3])$  ,  $G[5]$  ,  $\text{len}(G)$

## II. 3- Position valide d'une case

**Q 3-** Écrire la fonction **valide(i, j, G)**, qui reçoit en paramètres deux entiers **i** et **j**. La fonction renvoie **True**, si **i** et **j** sont positifs, et s'ils représentent, respectivement la ligne et la colonne d'une case qui existe dans la grille binaire **G**, sinon, la fonction renvoie **False**.

Exemples :

- La fonction **valide(0, 3, G)** renvoie : **True**
- La fonction **valide(7, 15, G)** renvoie : **False**
- La fonction **valide(-2, -8, G)** renvoie : **False**

## II. 4- Couleur d'une case

**Q 4-** Écrire la fonction **couleur(t, G)**, qui reçoit en paramètres un tuple **t** qui représente une case dans la grille binaire **G**. La fonction renvoie **1** si la couleur de la case **t** est blanche, sinon, elle renvoie **0**.

Exemples :

- La fonction **couleur((0, 3), G)** renvoie le nombre : **1**
- La fonction **couleur((0, 4), G)** renvoie le nombre : **0**

## II. 5- Cases voisines

*Dans une grille binaire, deux cases sont voisines, si elles ont la même couleur et si elles ont un seul côté en commun.*

	c	
a	b	e
	d	

Exemples :

- Les cases **a** et **b** sont voisines ;
- Les cases **b** et **c** sont voisines ;
- Les cases **b** et **d** sont voisines ;
- Les cases **b** et **e** ne sont pas voisines, (**b** et **e** n'ont pas la même couleur) ;
- Les cases **a** et **c** ne sont pas voisines, (**a** et **c** n'ont aucun côté en commun) ;
- Les cases **a** et **a** ne sont pas voisines (**a** et **a** n'ont pas un seul côté en commun).

**Q 5-** Écrire la fonction **list\_voisines(t, G)**, qui reçoit en paramètres un tuple **t** représentant une case dans la grille binaire **G**. La fonction renvoie la liste des cases voisines à la case **t**, dans la grille **G**.

Exemples :

- La fonction **list\_voisines((5, 4), G)** renvoie la liste : [ (4, 4), (6, 4), (5, 3), (5, 5) ]
- La fonction **list\_voisines((2, 4), G)** renvoie la liste : [ (2, 5) ]
- La fonction **list\_voisines((5, 1), G)** renvoie la liste : [ (4, 1), (5, 2) ]
- La fonction **list\_voisines((7, 2), G)** renvoie la liste : [ ]

## II. 6- Chemin dans une grille

On considère une liste **L** de tuples qui représentent des cases dans une grille binaire **G**.

La liste **L** est un **chemin** dans la grille **G**, si la liste **L** satisfait les **trois** conditions suivantes :

- c1** - La liste **L** contient au moins deux cases de la grille **G** ;
- c2** - Toutes les cases de **L** sont différentes deux à deux, (pas de doublon dans **L**) ;
- c3** - Deux cases consécutives dans **L** sont voisines.

**Exemples :**

- **L** = [ (0, 3), (0, 2), (0, 1), (0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (3, 2), (4, 2), (4, 3), (4, 4) ]  
La liste **L** est un chemin dans la grille **G**.
- **T** = [ (0, 0), (1, 1), (2, 1), (2, 2), (1, 2), (1, 1) ].  
La liste **T** n'est pas un chemin dans la grille **G**.

**Q 6-** Écrire la fonction **chemin(L, G)**, qui reçoit en paramètres une liste **L** de tuples représentant des cases dans la grille binaire **G**. La fonction renvoie **True** si la liste **L** est un chemin dans **G**, sinon, la fonction renvoie **False**.

## II. 7- Compression d'un chemin dans une grille binaire

La compression d'un chemin dans une grille binaire, consiste à remplacer, dans ce chemin, chaque suite d'au moins **trois** cases voisines qui se trouvent sur la même ligne ou bien sur la même colonne, par deux cases seulement : la première case et la dernière case.

**Exemples :**

- Cas d'une suite d'au moins 3 cases voisines, qui se trouvent sur la même ligne :

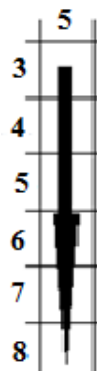


La suite des cases voisines : (2, 1), (2, 2), (2, 3), ..., (2, 8), sera remplacée par : (2, 1), (2, 8)



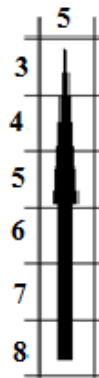
La suite des cases voisines : (2, 8), (2, 7), (2, 6), ..., (2, 1), sera remplacée par : (2, 8), (2, 1).

- Cas d'une suite d'au moins 3 cases voisines, qui se trouvent sur la même colonne :





La suite des cases : (3, 5), (4, 5), (5, 5), ..., (8, 5), sera remplacée par : (3, 5), (8, 5) ;



La suite des cases : (8, 5), (7, 5), (6, 5), ..., (3, 5), sera remplacée par : (8, 5), (3, 5).

**Q 7-** Écrire la fonction **compresse\_chemin(L)**, qui reçoit en paramètre une liste **L** qui contient un chemin dans une grille binaire. La fonction renvoie la liste qui contient le chemin compressé de **L**.

Exemples :

- **L** = [ (0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (5, 0), (6, 0), (7, 0), (8, 0), (9, 0), (9, 1), (9, 2) ]  
La fonction **compresse\_chemin(L)** renvoie la liste : [ (0, 0), (9, 0), (9, 2) ]
- **L** = [ (0, 0), (1, 0), (1, 1), (2, 1), (2, 0), (3, 0) ]  
La fonction **compresse\_chemin(L)** renvoie la liste : [ (0, 0), (1, 0), (1, 1), (2, 1), (2, 0), (3, 0) ]

## II. 8- Appartenance d'une case à un chemin compressé

**Q 8-** Écrire la fonction **appartient(t, L)**, qui reçoit en paramètres une liste **L** contenant un chemin compressé dans une grille binaire, et un tuple **t** représentant une case. La fonction renvoie **True** si le chemin compressé **L** passe par la case **t**, sinon, la fonction renvoie **False**.

Exemples :

On considère le chemin compressé, de l'exemple précédent : **L** = [ (0, 0) , (9, 0) , (9, 2) ]

- Les cases (0, 0), (1, 0), (2, 0), (8, 0), (9, 1) appartiennent au chemin **L** ;
- La case (0, 2) n'appartient pas au chemin **L**.

## II. 9- Longueur d'un chemin compressé

*Dans une grille binaire, la longueur d'un chemin compressé, est égale au nombre total des cases par lesquelles passe ce chemin.*

Exemples :

- La longueur du chemin compressé [ (0, 0), (9, 0), (9, 2) ] est : 12 ;
- La longueur du chemin compressé [ (0, 0), (1, 0), (1, 1), (2, 1), (2, 0), (3, 0), (3, 1) ] est : 7.

**Q 9-** Écrire la fonction, de complexité linéaire, **longueur\_chemin(L)**, qui reçoit en paramètre une liste **L** qui contient un chemin compressé. La fonction renvoie la longueur du chemin **L**.

## II. 10- Chemin entre deux cases

On suppose que **a** et **b** sont deux tuples qui représentent deux cases distinctes, de même couleur, dans une grille binaire **G**.

Un chemin entre les cases **a** et **b**, s'il existe, est un chemin compressé, tel que le tuple **a** est son premier élément, et le tuple **b** est son dernier élément.

**NB** : On peut trouver plusieurs chemins entre les cases **a** et **b**.

### Exemple :

Les chemins suivants sont des chemins compressés entre les cases **a = (0, 3)** et **b = (4, 4)** :

- [ **(0, 3)**, (0, 2), (1, 2), (1, 1), (0, 1), (0, 0), (3, 0), (3, 2), (4, 2), (4, 3), (6, 3), (6, 4), **(4, 4)** ]
- [ **(0, 3)**, (0, 2), (1, 2), (1, 1), (3, 1), (3, 2), (4, 2), **(4, 4)** ]
- [ **(0, 3)**, (0, 2), (1, 2), (1, 1), (2, 1), (2, 0), (3, 0), (3, 2), (4, 2), **(4, 4)** ]
- [ **(0, 3)**, (0, 0), (1, 0), (1, 1), (3, 1), (3, 2), (4, 2), (4, 3), (6, 3), (6, 4), **(4, 4)** ]
- ...

**Q 10. a-** Écrire la fonction **chemins(G, a, b, chemin)**, reçoit en paramètres deux tuples **a** et **b** distincts qui représentent deux cases de même couleur, dans la grille binaire **G**. Le paramètre **chemin** est une liste initialisée par la liste vide. Cette fonction renvoie une nouvelle liste contenant tous les chemins entre la case **a** et la case **b** dans la grille **G**.

### Exemples :

- La fonction **chemins(G, (0, 3), (4, 4), [ ])** renvoie la liste des chemins suivants :  
[ [ **(0, 3)**, (0, 2), (1, 2), (1, 1), (2, 1), (3, 1), (3, 2), (4, 2), (4, 3), **(4, 4)** ], [ **(0, 3)**, (0, 2), (1, 2), (1, 1), (0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (3, 1), (3, 2), (4, 2), (4, 3), (5, 3), (5, 4), **(4, 4)** ], [ **(0, 3)**, (0, 2), (1, 2), (1, 1), (2, 1), (3, 1), (3, 2), (4, 2), (4, 3), (5, 3), (5, 4), **(4, 4)** ], [ **(0, 3)**, (0, 2), (1, 2), (1, 1), (0, 1), (0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (3, 2), (4, 2), (4, 3), **(4, 4)** ] , ... ]
- La fonction **chemins(G, (6, 0), (7, 2), [ ])** renvoie la liste : [ ]

**Q 10. b-** Écrire la fonction **list\_chemins(G, a, b)**, reçoit en paramètres deux tuples **a** et **b** qui représentent deux cases quelconques dans la grille binaire **G**. Cette fonction renvoie la liste de tous les chemins compressés entre la case **a** et la case **b**, dans la grille **G**.

### Exemples :

- La fonction **list\_chemins(G, (0, 3), (4, 4), [ ])** renvoie la liste des chemins compressés suivants :  
[ [ **(0, 3)**, (0, 2), (1, 2), (1, 1), (3, 1), (3, 2), (4, 2), **(4, 4)** ], [ **(0, 3)**, (0, 2), (1, 2), (1, 1), (0, 1), (0, 0), (2, 0), (2, 1), (3, 1), (3, 2), (4, 2), (4, 3), (5, 3), (5, 4), **(4, 4)** ], [ **(0, 3)**, (0, 2), (1, 2), (1, 1), (2, 1), (3, 1), (3, 2), (4, 2), (4, 3), (5, 3), (5, 4), **(4, 4)** ], [ **(0, 3)**, (0, 2), (1, 2), (1, 1), (0, 1), (0, 0), (3, 0), (3, 2), (4, 2), **(4, 4)** ] , ... ]
- La fonction **list\_chemins(G, (6, 0), (8, 1), [ ])** renvoie la liste : [ ]

## II. 11- Tri d'une liste de chemins

**Q 11-** Écrire la fonction **tri\_chemins(R)**, qui reçoit en paramètre une liste **R** contenant tous les chemins compressés entre deux cases dans une grille binaire. La fonction trie les chemins compressés de **R** dans l'ordre croissant des longueurs des chemins. La longueur d'un chemin telle qu'elle est définie dans la question **II. 9**

**NB** : Ne pas utiliser la méthode `sort()`, ni la fonction `sorted()`.

**Exemple :**

On considère la liste **R** suivante, qui contient **3** chemins compressés dans la grille binaire **G**.

**R** = [ [ (0, 3), (0, 0), (3, 0), (3, 2), (4, 2), (4, 4) ] , [ (0, 3), (0, 2), (1, 2), (1, 1), (3, 1), (3, 2), (4, 2), (4, 4) ] , [ (0, 3), (0, 1), (3, 1), (3, 2), (4, 2), (4, 4) ] ]

Après l'appel de la fonction **tri\_chemins (R)**, on obtient la liste triée suivante :

[ [ (0, 3), (0, 2), (1, 2), (1, 1), (3, 1), (3, 2), (4, 2), (4, 4) ] , [ (0, 3), (0, 1), (3, 1), (3, 2), (4, 2), (4, 4) ] , [ (0, 3), (0, 0), (3, 0), (3, 2), (4, 2), (4, 4) ] ]

## **II. 12- Plus courts chemins entre deux cases**

***a** et **b** sont deux tuples qui représentent deux cases dans une grille binaire **G**. Le **plus court chemin** entre **a** et **b** est le chemin compressé entre **a** et **b** ayant la plus petite longueur.*

***NB** : On peut trouver plusieurs plus courts chemins entre les cases **a** et **b**.*

**Q 12-** Écrire la fonction **plus\_court\_chemins (G, a, b)**, qui renvoie la liste de tous les plus courts chemins compressés entre deux cases **a** et **b**.

**Exemples :**

- La fonction **plus\_court\_chemins ((0, 3), (4, 4), G)** renvoie la liste des deux chemins suivants :  
[ [(0, 3), (0, 2), (1, 2), (1, 1), (3, 1), (3, 2), (4, 2), (4, 4)] , [(0, 3), (0, 1), (3, 1), (3, 2), (4, 2), (4, 4)] ]
- La fonction **plus\_court\_chemins ((6, 0), (7, 2), G)** renvoie la liste : [ ]

## **II. 13- Zone d'une case dans une grille binaire**

*On considère un tuple **t** qui représente une case dans une grille binaire **G**.*

*La zone de la case **t** est l'ensemble qui contient la case **t**, et toutes les cases de la grille binaire **G**, qu'on peut joindre par un chemin à partir de la case **t**.*

**Exemples :**

- La zone de la case **(8, 2)** est l'ensemble composé des cases suivantes :  
**(8, 2)** , (7, 3) , (8, 3) , (7, 1) , (8, 1) , (9, 3) , (7, 4) , (9, 4)
- La zone de la case **(9, 6)** est l'ensemble composé des cases suivantes :  
**(9, 6)** , (7, 9) , (8, 4) , (6, 6) , (7, 6) , (9, 8) , (9, 9) , (8, 9) , (7, 5) , (8, 8) , (8, 6) , (9, 5), (8, 5), (9, 7)
- La zone de la case **(7, 2)** est l'ensemble composé d'une seule cases : **(7, 2)**

**Q 13-**Écrire la fonction **compte\_zones (G)**, qui reçoit en paramètre une grille binaire **G**, et qui renvoie le nombre de zones dans la grille binaire **G**.

**Exemple :**

La fonction **compte\_zone (G)** renvoie le nombre : **10**

~~~~~ **FIN DE L'ÉPREUVE** ~~~~~

Epreuve d'Informatique – Session 2019 – Filière PSI

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Remarques générales :

- Cette épreuve est composée d'un exercice et d'un problème indépendants ;
- Toutes les instructions et les fonctions demandées seront écrites en Python ;
- Les questions non traitées peuvent être admises pour aborder les questions ultérieures ;
- Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

~~~~~

Exercice : (4 points)

Somme et factoriel

- 1 pt

**Q1-** Écrire la fonction **factoriel (k)** qui reçoit en paramètre un entier positif **k** et qui renvoie la valeur du factoriel de **k** :  $k! = 1 * 2 * 3 * \dots * k$ .

Exemples :

  - La fonction **factoriel (5)** renvoie le nombre :  $120 = 1 * 2 * 3 * 4 * 5$
  - La fonction **factoriel (0)** renvoie le nombre : **1**
- 0.5 pt

**Q2-** Déterminer la complexité de la fonction **factoriel (k)**, et justifier votre réponse.
- 1.5 pt

**Q3-** Écrire la fonction **som\_fact (L)** qui reçoit en paramètre une liste **L** de nombres entiers positifs. La fonction renvoie la somme des factoriels des éléments de **L**.

Exemple :

**L = [ 5, 3, 0, 6, 1 ]**

La fonction **som\_fact (L)** renvoie la valeur de la somme :  $5! + 3! + 0! + 6! + 1!$
- 1 pt

**Q4-** Déterminer la complexité de la fonction **som\_fact (L)**, et justifier votre réponse.
- ~~~~~

## Problème :

### *Réseau routier*

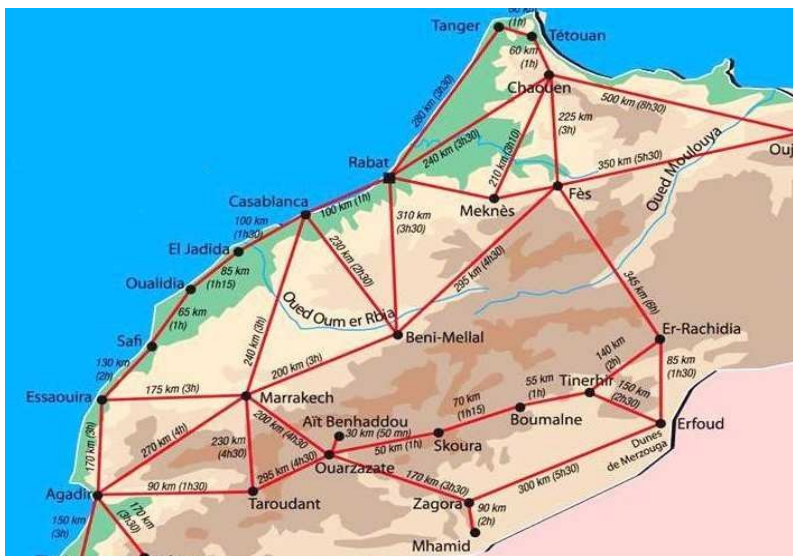


Figure 1 : Extrait du réseau routier du Maroc

Un réseau routier peut être représenté par un dessin qui se compose de points et de traits continus reliant deux à deux certains de ces points : les **points** sont les **villes**, et les **lignes** sont les **routes**. On considère que toutes les routes sont à double sens. Chaque route peut être affectée par une valeur qui peut représenter le temps ou la distance entre deux villes, ...

Étant donné un réseau routier, on pourra s'intéresser à la résolution des problèmes suivants :

- Quel est le plus court chemin entre deux villes ?
- Entre deux villes, quel est le nombre de chemins passant par un nombre de routes donné ?
- Est-il possible de passer par toutes les villes du réseau, sans passer deux fois par une même ville ?, si oui, quel est le plus court chemin ?
- Entre deux villes, quel est le chemin ayant le moindre coût ?
- ...

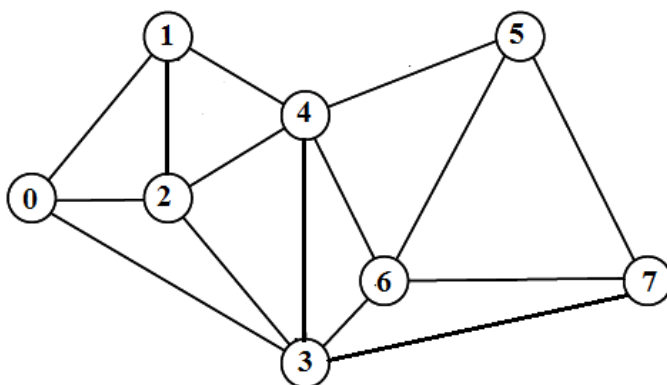
## Partie I :

### *Modélisation d'un réseau routier*

On considère un réseau routier composé de  $n$  villes (avec  $n \geq 2$ ). Les villes du réseau routier sont numérotées par des entiers allant de  $0$  à  $n-1$ .

**Exemple :**





**Figure 2** : Réseau routier composé de 15 routes, et de 8 villes numérotées de 0 à 7.

Pour plus de clarté, tous les exemples de ce problème seront appliqués sur le réseau routier de la **figure 2**.

### Représentation du réseau routier

Pour représenter un réseau routier de  $n$  villes, on utilise une matrice symétrique  $R$  d'ordre  $n$  ( $n$  lignes et  $n$  colonnes), telle que :

Pour toutes les villes  $i$  et  $j$ , telles que  $0 \leq i < n$  et  $0 \leq j < n$ , on a :

$R[i, j] = R[j, i] = 1$ , s'il existe une route qui relie entre la ville  $i$  et la ville  $j$  ;

$R[i, j] = R[j, i] = 0$ , sinon.

#### Exemple :

Le réseau routier de la **figure 2** est représenté par la matrice symétrique  $R$ , d'ordre 8, suivante :

$i/j$	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0
1	1	0	1	0	1	0	0	0
2	1	1	0	1	1	0	0	0
3	1	0	1	0	1	0	1	1
4	0	1	1	1	0	1	1	0
5	0	0	0	0	1	0	1	1
6	0	0	0	1	1	1	0	1
7	0	0	0	1	0	1	1	0

**NB** : Dans la matrice symétrique  $R$ , les lignes  $i$  et les colonnes  $j$  représentent les villes du réseau routier.

### I. 1- Représentation de la matrice du réseau routier en Python

Pour représenter la matrice symétrique  $R$  d'ordre  $n$ , on utilise une liste composée de  $n$  listes qui sont toutes de même longueur  $n$ .

Exemple :

La matrice symétrique **R**, du réseau routier de la **figure 2**, est représentée par la liste **R**, composée de 8 listes, de taille 8 chacune :

```
R = [ [0, 1, 1, 1, 0, 0, 0, 0], [1, 0, 1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 1, 0, 0, 0],  
      [1, 0, 1, 0, 1, 0, 1, 1], [0, 1, 1, 1, 0, 1, 1, 0], [0, 0, 0, 0, 1, 0, 1, 1],  
      [0, 0, 0, 1, 1, 1, 0, 1], [0, 0, 0, 1, 0, 1, 1, 0]  
    ]
```

**R[i][j]** est l'élément de **R**, à la **i<sup>ème</sup>** ligne et la **j<sup>ème</sup>** colonne.

**Exemples :** **R[0][0]** est l'élément : 0, et **R[0][2]** est l'élément : 1

**R[i]** est la ligne d'indice **i** dans **R**.

**Exemple :** **R[0]** est la liste [ 0 , 1 , 1 , 1 , 0 , 0 , 0 , 0 ], qui représente la ligne d'indice 0 dans la matrice **R**.

**Q 1-** À partir de la matrice symétrique **R** de la **figure 2**, donner les résultats des expressions suivantes :

**R[4][2]** , **R[1]** , **len(R[2])** , **len(R)**

## **I. 2- Villes voisines**

*i et j sont deux villes dans un réseau routier représenté par une matrice symétrique **R**.*

*Les villes **i** et **j** sont **voisines**, s'il existe une route entre la ville **i** et la ville **j**.*

**Q 2. a-** Écrire la fonction **voisines(i, j, R)**, qui reçoit en paramètres deux villes **i** et **j** d'un réseau routier représenté par la matrice symétrique **R**. La fonction renvoie **True** si les villes **i** et **j** sont voisines, sinon, la fonction renvoie **False**.

Exemples :

- La fonction **voisines(3, 0, R)** renvoie **True** ; (les villes 3 et 0 sont voisines)
- La fonction **voisines(3, 5, R)** renvoie **False**. (les villes 3 et 5 ne sont pas voisines)

**Q 2. b-** Écrire la fonction **list\_voisines(i, R)**, qui reçoit en paramètres une ville **i** d'un réseau routier représenté par la matrice symétrique **R**. La fonction renvoie la liste de toutes les villes voisines à la ville **i**.

Exemple :

La fonction **list\_voisines(3, R)** renvoie la liste des villes voisines à la ville 3 : [ 0 , 2 , 4 , 6 , 7 ]

## **I. 3- Degré d'une ville**

*Dans un réseau routier, le **degré** d'une ville **i** est le nombre de villes voisines à la ville **i**.*

**Q 3-** Écrire la fonction **degre(i, R)**, qui reçoit en paramètres une ville **i** d'un réseau routier représenté par la matrice symétrique **R**. La fonction renvoie le degré de la ville **i**.

Exemples :

- La fonction **degre(3, R)** renvoie le nombre : 5 (La ville 3 possède 5 villes voisines)
- La fonction **degre(0, R)** renvoie le nombre : 3 (La ville 0 possède 3 villes voisines)
- La fonction **degre(2, R)** renvoie le nombre : 4 (La ville 2 possède 4 villes voisines)

### I. 4- Liste des degrés des villes

**Q 4.** Écrire la fonction **liste\_degres (R)**, qui reçoit en paramètre la matrice symétrique **R** représentant un réseau routier. La fonction renvoie une liste **D** contenant des tuples. Chaque tuple de **D** est composé de deux éléments : une **ville** du réseau routier, et le **degré** de cette ville.

Exemple :

La fonction **liste\_degres (R)** renvoie la liste **D** suivante :

**D = [ (0 , 3) , (1 , 3) , (2 , 4) , (3 , 5) , (4 , 5) , ( 5, 3) , (6 , 4) , (7 , 3) ]**

### I. 5- Tri des villes

**Q 5. a-** Écrire la fonction **tri\_degres (D)**, qui reçoit en paramètre la liste **D** des degrés des villes. La fonction trie les tuples de la liste **D** dans l'ordre décroissant des degrés des villes.

Exemple :

**D = [ (0 , 3) , (1 , 3) , (2 , 4) , (3 , 5) , (4 , 5) , ( 5, 3) , (6 , 4) , (7 , 3) ]**

Après l'appel de la fonction **tri\_degres (D)**, on obtient la liste suivante :

**[ (3 , 5) , (4 , 5) , (2 , 4) , (6 , 4) , (0 , 3) , (5 , 3) , (1 , 3) , (7 , 3) ]**

**Q 5. b-** Écrire la fonction **tri\_villes (R)**, qui reçoit en paramètre la matrice symétrique **R** représentant un réseau routier. La fonction renvoie la liste des villes triées dans l'ordre décroissant des degrés des villes.

Exemple :

La fonction **tri\_villes (R)** renvoie la liste des villes triées dans l'ordre décroissant des degrés :

**[ 3 , 4 , 2 , 6 , 0 , 5 , 1 , 7 ]**

## Partie II :

### *Coloration optimale des villes d'un réseau routier*

Une **coloration** des villes du réseau routier est une affectation de couleurs à chaque ville, de façon à ce que deux villes voisines soient affectées par deux couleurs différentes. Le nombre minimum de couleurs nécessaires pour colorier un réseau routier est appelé : **nombre chromatique**.

La recherche du nombre chromatique est une question qui intervient dans beaucoup de problèmes concrets :

- L'établissement d'emplois du temps ;
- la gestion de ressources partagées ;
- la compilation dans l'utilisation efficace des registres ;
- ...

On cherche à construire une liste **C** qui contiendra les couleurs des villes. Ces couleurs seront représentées par des entiers strictement positifs : *chaque élément **C[k]** contiendra la couleur de la ville **k** du réseau routier.*

Pour construire la liste **C** des couleurs, on propose d'utiliser un algorithme, appelé : **algorithme de glouton**. C'est un algorithme couramment utilisé dans la résolution de ce genre de problèmes, afin d'obtenir des solutions optimales.

**Principe de l'algorithme de glouton :**

**V** est la liste des villes triées dans l'ordre décroissant des degrés des villes

**C** est une liste de même taille que **V**, initialisée par des 0

Pour toute ville **k** de **V** :

**C[k]** première couleur non utilisée par les villes voisines à la ville **k**

Retourner la liste **C**

**II. 6- Premier entier qui n'existe pas dans une liste d'entiers**

**Q 6-** Écrire la fonction **premier\_entier(L)**, qui reçoit en paramètre une liste **L** de nombres entiers positifs. La fonction renvoie le premier entier positif qui n'appartient pas à la liste **L**.

**Exemple :**

La fonction **premier\_entier ( [ 0 , 0 , 3 , 1 , 0 , 1 , 0 , 0 ] )** renvoie le nombre : 2

**II. 7- Liste des couleurs des villes voisines à une ville**

**Q 7-** Écrire la fonction **couleurs\_voisines(k,C,R)**, qui reçoit en paramètres une ville **k** d'un réseau routier représenté par la matrice symétrique **R**, et la liste **C** des couleurs des villes du réseau. La fonction renvoie la liste des **C[i]** telle que **i** est une ville voisine à la ville **k**.

**Exemples :**

- La fonction **couleurs\_voisines (4, [ 0 , 0 , 0 , 1 , 0 , 0 , 0 , 0 ], R)** renvoie la liste : **[0, 0, 1, 0, 0]**
- La fonction **couleurs\_voisines (2, [ 0 , 0 , 0 , 1 , 2 , 0 , 0 , 0 ], R)** renvoie la liste: **[0, 0, 1, 2]**
- La fonction **couleurs\_voisines (6, [ 0 , 0 , 3 , 1 , 2 , 0 , 0 , 0 ], R)** renvoie la liste: **[1, 2, 0, 0]**
- La fonction **couleurs\_voisines (0, [ 0 , 0 , 3 , 1 , 2 , 0 , 3 , 0 ], R)** renvoie la liste: **[0, 3, 1]**

**II. 8- Coloration des villes**

**Q 8-** Écrire la fonction **couleurs\_villes(R)**, qui reçoit en paramètre la matrice symétrique **R** représentant un réseau routier. La fonction renvoie la liste **C** des couleurs des villes, en utilisant le principe de glouton cité ci-dessus.

**Exemple :**

La fonction **couleurs\_villes (R)** renvoie la liste des couleurs : **C = [ 2 , 1 , 3 , 1 , 2 , 1 , 3 , 2 ]**

Dans cette liste **C** :

- Les villes **0, 4** et **7** ont la même couleur : **2** ;
- Les villes **1, 3** et **5** ont la même couleur : **1** ;
- Les villes **2** et **6** ont la même couleur : **3**.

*NB : Dans cet exemple, le nombre chromatique est 3.*

## Partie III :

### *Chemins dans un réseau routier*

#### III. 9- Chemin entre deux villes

Dans un réseau routier, un **chemin** est un tuple **T** qui contient des villes du réseau, et qui satisfait les deux conditions suivantes :

- c1** : Le tuple **T** contient au moins deux villes du réseau routier ;
- c2** : Deux villes consécutives dans **T** sont voisines.

#### Exemples :

- Le tuple ( 2, 0, 1, 4, 3 ) est un chemin entre la ville 2 et la ville 3, composé de 4 routes ;
- Le tuple ( 7, 6, 4, 5, 6, 3, 4 ) est un chemin entre la ville 7 et la ville 4, composé de 6 routes ;
- Le tuple ( 3, 1, 4, 7 ) n'est pas un chemin.

**Q 9-** Écrire la fonction **chemin\_valide(T,R)**, qui reçoit en paramètres un tuple **T** contenant des villes du réseau routier représenté par la matrice symétrique **R**. La fonction renvoie **True** si le tuple **T** satisfait les deux conditions **c1** et **c2** citées ci-dessus, sinon, la fonction renvoie **False**.

#### III. 10- Chemin simple entre deux villes

Dans un réseau routier, un **chemin simple** est un chemin qui passe une seule fois par la même ville.

#### Exemples :

- Le chemin ( 2, 0, 1, 4, 3 ) est un chemin simple entre la ville 2 et la ville 3 ;
- Le chemin ( 6, 7, 3, 4, 2, 3, 6, 5 ) n'est pas simple.

**Q 10-** Écrire la fonction **chemin\_simple(T,R)**, qui reçoit en paramètres un chemin **T** dans un réseau routier représenté par la matrice symétrique **R**. La fonction renvoie **True** si le chemin **T** est un simple, sinon, la fonction renvoie **False**.

#### III. 11- Plus court chemin entre deux villes

On suppose que la fonction **liste\_chemins(i,j,R)**, reçoit en paramètres deux villes **i** et **j** d'un réseau routier représenté par la matrice symétrique **R**. Cette fonction renvoie une liste qui contient tous les chemins simples entre la ville **i** et la ville **j**.

#### Exemple :

La fonction **liste\_chemins(1,5,R)** renvoie la liste de tous les chemins simples entre les villes 1 et 5 :

[ (1, 0, 2, 3, 4, 5) , (1, 0, 2, 3, 4, 6, 5) , (1, 0, 2, 3, 4, 6, 7, 5) , (1, 0, 3, 4, 6, 7, 5) ,  
(1, 0, 3, 6, 4, 5) , (1, 0, 3, 6, 5) , (1, 0, 3, 6, 7, 5) , (1, 2, 0, 3, 6, 5) , (1, 2, 0, 3, 6, 7, 5) ,  
(1, 2, 0, 3, 7, 5) , (1, 2, 0, 3, 7, 6, 4, 5) , (1, 2, 3, 7, 6, 5) , (1, 4, 2, 0, 3, 7, 5) , (1, 4, 2, 0,  
3, 7, 6, 5) , (1, 4, 2, 3, 6, 5) , (1, 4, 2, 3, 6, 7, 5) , (1, 4, 2, 3, 7, 5) , (1, 4, 2, 3, 7, 6, 5) , (  
1, 4, 3, 6, 5) , (1, 4, 3, 6, 7, 5) , (1, 4, 3, 7, 5) , (1, 4, 3, 7, 6, 5) , (1, 4, 5) , (1, 4, 6, 3, 7,  
5) , (1, 4, 6, 5) , ... , (1, 4, 6, 7, 5) ]



Le **plus court chemin** entre une ville  $i$  et une ville  $j$  est un chemin simple entre la ville  $i$  et la ville  $j$ , et qui traverse le moins de villes.

**NB** : On peut trouver plusieurs plus courts chemins entre deux villes.

**Q 11-** Écrire la fonction **pc\_chemins**( $i, j, R$ ), qui reçoit en paramètres deux villes  $i$  et  $j$  d'un réseau routier représenté par la matrice symétrique  $R$ . Cette fonction renvoie la liste de tous les plus courts chemins entre la ville  $i$  et la ville  $j$ .

**Exemples** :

- La fonction **pc\_chemins**( $0, 7, R$ ) renvoie la liste [ ( $0, 3, 7$ ) ] ;
- La fonction **pc\_chemins**( $7, 1, R$ ) renvoie la liste :  
[ ( $7, 3, 0, 1$ ) , ( $7, 3, 2, 1$ ) , ( $7, 3, 4, 1$ ) , ( $7, 5, 4, 1$ ) , ( $7, 6, 4, 1$ ) ].

**Épreuve d'Informatique – Session 2019 – Filière TSI**

*Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.*

**Remarques générales :**

- Cette épreuve est composée d'un exercice et de deux parties tous indépendants ;
- Toutes les instructions et les fonctions demandées seront écrites en Python ;
- Les questions non traitées peuvent être admises pour aborder les questions ultérieures ;
- Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

~~~~~

Exercice : (4 points)***Somme et puissance***

- 1 pt** **Q1-** Écrire la fonction **somme (L)** qui reçoit en paramètre une liste **L** de nombres entiers, et qui retourne la somme des éléments de **L**.

Exemple :

La fonction **somme ([7, 0, -1, 5, -3])** renvoie la valeur **8 = 7 + 0 + (-1) + 5 + (-3)**

- 0.5 pt** **Q2-** Déterminer la complexité de la fonction **somme (L)**, et justifier votre réponse.

- 1 pt** **Q3-** Écrire la fonction **list_puissances (L, p)** qui reçoit en paramètres une liste **L** de nombres entiers, et un entier **p** strictement positif. La fonction renvoie une nouvelle liste qui contient les éléments de **L** élevés chacun à la puissance **p**.

Exemple :

La fonction **list_puissances ([7, 0, -1, 5, -3], 2)** renvoie la liste **[49, 0, 1, 25, 9]**

- 1 pt** **Q4-** Écrire la fonction **som_puiss (L, p)** qui reçoit en paramètres une liste **L** de nombres entiers, et un entier **p** strictement positif. La fonction renvoie la somme des éléments de **L** élevés chacun à la puissance **p**.

Exemple :

La fonction **som_puiss ([7, 0, -1, 5, -3], 2)** renvoie le nombre **84 = 7² + 0² + (-1)² + 5² + (-3)²**

- 0.5 pt** **Q5-** Déterminer la complexité de la fonction **som_puiss (L, p)**, et justifier votre réponse.

Partie II :

Réseau routier



Figure 1 : Extrait du réseau routier du Maroc

Un réseau routier peut être représenté par un dessin qui se compose de points et de traits continus reliant deux à deux certains de ces points : les **points** sont les **villes**, et les **lignes** sont les **routes**. On considère que toutes les routes sont à double sens. Chaque route peut être affectée par une valeur qui peut représenter le temps ou la distance entre deux villes, ...

Étant donné un tel réseau, on pourra s'intéresser, par exemple, à la résolution des problèmes suivants :

Quel est le plus court chemin entre deux villes ?

Entre deux villes, quel est le nombre de chemins passant par un nombre de routes donné ?

Est-il possible de passer par toutes les villes du réseau, sans passer deux fois par une même ville ?, si oui, quel est le plus court chemin ?

Entre deux villes, quel est le chemin ayant le moindre coût ?

...

Modélisation d'un réseau routier

On considère un réseau routier composé de n villes (avec $n \geq 2$). Les villes du réseau routier sont numérotées par des entiers allant de 0 à $n-1$.

Exemple :

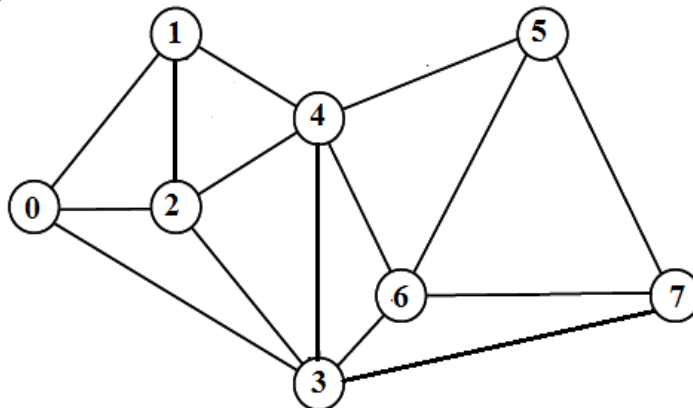


Figure 2 : Réseau routier composé de 15 routes, et de 8 villes numérotées de 0 à 7.

Pour plus de clarté, tous les exemples de cette partie seront appliqués sur le réseau routier de la **figure 2**.

Représentation du réseau routier par une matrice

Pour représenter un réseau routier de n villes, on utilise une matrice symétrique R d'ordre n (n lignes et n colonnes), telle que :

Pour toutes les villes i et j , telles que $0 \leq i < n$ et $0 \leq j < n$, on a :

$R[i, j] = R[j, i] = 1$, s'il existe une route qui relie entre la ville i et la ville j ;
 $R[i, j] = R[j, i] = 0$, sinon.

Exemple :

Le réseau routier de la **figure 2** est représenté par la matrice symétrique R suivante :

| i/j | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|-------|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 2 | 1 | 1 | 0 | 1 | 1 | 0 | 0 | 0 |
| 3 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |
| 4 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 0 |
| 5 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 |
| 6 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 1 |
| 7 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 |

NB : Dans la matrice symétrique R , les lignes i et les colonnes j représentent les villes du réseau routier.

II. 1- Représentation de la matrice du réseau routier par une liste de listes

Pour représenter la matrice symétrique R d'ordre n , on utilise une liste composée de n listes toutes de même longueur n .

Exemple :

La matrice symétrique R , du réseau routier de la **figure 2**, est représentée par la liste R , composée de **8** listes, de taille **8** chacune :

$R = [$
 $[0, 1, 1, 1, 0, 0, 0, 0],$
 $[1, 0, 1, 0, 1, 0, 0, 0],$
 $[1, 1, 0, 1, 1, 0, 0, 0],$
 $[1, 0, 1, 0, 1, 0, 1, 1],$
 $[0, 1, 1, 1, 0, 1, 1, 0],$
 $[0, 0, 0, 0, 1, 0, 1, 1],$
 $[0, 0, 0, 1, 1, 1, 0, 1],$
 $[0, 0, 0, 1, 0, 1, 1, 0]$
 $]$

$R[i][j]$ est l'élément de R , à la $i^{\text{ème}}$ ligne et la $j^{\text{ème}}$ colonne.

Exemples : $R[0][0]$ est l'élément : 0, et $R[0][2]$ est l'élément : 1

$R[i]$ est la ligne d'indice i dans R .

Exemple : $R[0]$ est la liste : [0 , 1 , 1 , 1 , 0 , 0 , 0 , 0], qui représente la ligne d'indice 0 dans R .

Q 1- À partir de la matrice R , donner les résultats des expressions suivantes :

$R[4][2]$, $R[1]$, $\text{len}(R[2])$, $\text{len}(R)$

II. 2- Villes voisines

i et j sont deux villes dans un réseau routier représenté par une matrice symétrique R .

Les villes i et j sont **voisines**, s'il existe une route entre la ville i et la ville j .

Q 2. a- Écrire la fonction **voisines** (i, j, R), qui reçoit en paramètres une ville i d'un réseau routier représenté par la matrice symétrique R . La fonction renvoie **True** si les villes i et j sont voisines, sinon, la fonction renvoie **False**.

Exemples :

- La fonction **voisines** (3, 0, R) renvoie **True** ; (les villes 3 et 0 sont voisines)
- La fonction **voisines** (3, 5, R) renvoie **False**. (les villes 3 et 5 ne sont pas voisines)

Q 2. b- Écrire la fonction **list_voisines** (i, R), qui reçoit en paramètres une ville i d'un réseau routier représenté par la matrice symétrique R . La fonction renvoie la liste de toutes les villes voisines à la ville i .

Exemple :

La fonction **list_voisines** (3, R) renvoie la liste : [0 , 2 , 4 , 6 , 7]

II. 3- Degré d'une ville

Dans un réseau routier, le **degré** d'une ville i est le nombre de villes voisines à la ville i .

Q 3- Écrire la fonction **degre** (i, R), qui reçoit en paramètres une ville i d'un réseau routier représenté par la matrice symétrique R . La fonction renvoie le degré de la ville i .

Exemples :

- La fonction **degre** (3, R) renvoie le nombre : 5 (La ville 3 possède 5 villes voisines)
- La fonction **degre** (0, R) renvoie le nombre : 3 (La ville 0 possède 3 villes voisines)
- La fonction **degre** (2, R) renvoie le nombre : 4 (La ville 2 possède 4 villes voisines)

II. 4- Liste des degrés des villes

Q 4- Écrire la fonction **list_degrees** (R), qui reçoit en paramètres la matrice symétrique R qui représente un réseau routier. La fonction renvoie une liste contenant des tuples. Chaque tuple est composé de deux éléments : une ville du réseau routier, et le **degré** de cette ville.

Exemple :

La fonction **list_degrees (R)** renvoie la liste **D** suivante :

D = [(0, 3), (1, 3), (2, 4), (3, 5), (4, 5), (5, 3), (6, 4), (7, 3)]

II. 5- Tri des villes

Q 5- Écrire la fonction **tri_degrees (D)**, qui reçoit en paramètre la liste **D** des degrés des villes. La fonction trie les tuples de la liste **D** dans l'ordre décroissant des degrés des villes.

Exemple :

D = [(0, 3), (1, 3), (2, 4), (3, 5), (4, 5), (5, 3), (6, 4), (7, 3)]

Après l'appel de la fonction **tri_degrees (D)**, on obtient la liste suivante :

[(3, 5), (4, 5), (2, 4), (6, 4), (0, 3), (5, 3), (1, 3), (7, 3)]

II. 6- Chemin entre deux villes

Dans un réseau routier, un **chemin** est un tuple **T** qui contient des villes du réseau, et qui satisfait les deux conditions suivantes :

- c1** : Le tuple **T** contient au moins deux villes du réseau routier ;
- c2** : Deux villes consécutives dans **T** sont voisines.

Exemples :

- Le tuple **(2, 0, 1, 4, 3)** est un chemin entre la ville **2** et la ville **3**, composé de **4** routes ;
- Le tuple **(7, 6, 4, 5, 6, 3, 4)** est un chemin entre la ville **7** et la ville **4**, composé de **6** routes ;
- Le tuple **(3, 1, 4, 6)** n'est pas un chemin.

Q 6- Écrire la fonction **chemin_valide (T, R)**, qui reçoit en paramètre un tuple **T** contenant des villes du réseau routier représenté par la matrice symétrique **R**. La fonction renvoie **True** si le tuple **T** satisfait les deux conditions **c1** et **c2** citées ci-dessus, sinon, la fonction renvoie **False**.

II. 7- Chemin simple entre deux villes

*Dans un réseau routier, un **chemin simple** est un chemin qui passe une seule fois par la même ville.*

Exemples :

- Le chemin **(2, 0, 1, 4, 3)** est un chemin simple ;
- Le chemin **(6, 7, 3, 4, 2, 3, 6, 5)** n'est pas un chemin simple.

Q 7- Écrire la fonction **chemin_simple (T, R)**, qui reçoit en paramètres un chemin **T** dans un réseau routier représenté par la matrice symétrique **R**. La fonction renvoie **True** si le chemin **T** est **un** simple, sinon, la fonction renvoie **False**.

II. 8- Plus court chemin entre deux villes

On suppose que la fonction **liste_chemins (i, j, R)**, reçoit en paramètres deux villes **i** et **j** d'un réseau routier représenté par la matrice symétrique **R**. Cette fonction renvoie une liste qui contient tous les chemins simples entre la ville **i** et la ville **j**.

Exemple :

La fonction **liste_chemins (1, 5, R)** renvoie la liste de tous les chemins simples entre les villes **1** et **5** :

[(1, 0, 2, 3, 4, 5) , (1, 0, 2, 3, 4, 6, 5) , (1, 0, 2, 3, 4, 6, 7, 5) , (1, 0, 3, 4, 6, 7, 5) ,
(1, 0, 3, 6, 4, 5) , (1, 0, 3, 6, 5) , (1, 0, 3, 6, 7, 5) , (1, 2, 0, 3, 6, 5) , (1, 2, 0, 3, 6, 7, 5) ,
(1, 2, 0, 3, 7, 5) , (1, 2, 0, 3, 7, 6, 4, 5) , (1, 2, 3, 7, 6, 5) , (1, 4, 2, 0, 3, 7, 5) , (1, 4, 2, 0,
3, 7, 6, 5) , (1, 4, 2, 3, 6, 5) , (1, 4, 2, 3, 6, 7, 5) , (1, 4, 2, 3, 7, 5) , (1, 4, 2, 3, 7, 6, 5) , (
1, 4, 3, 6, 5) , (1, 4, 3, 6, 7, 5) , (1, 4, 3, 7, 5) , (1, 4, 3, 7, 6, 5) , (1, 4, 5) , (1, 4, 6, 3, 7,
5) , (1, 4, 6, 5) , ... , (1, 4, 6, 7, 5)]

*Le **plus court chemin** entre une ville **i** et une ville **j** est un chemin simple entre la ville **i** et la ville **j**, et qui traverse le moins de villes.*

NB : On peut trouver plusieurs plus courts chemins entre deux villes.

Q 8- Écrire la fonction **pc_chemins (i, j, R)**, qui reçoit en paramètres deux villes **i** et **j** d'un réseau routier représenté par la matrice symétrique **R**. Cette fonction renvoie la liste de tous les plus courts chemins entre la ville **i** et la ville **j**.

Exemples :

- La fonction **pc_chemins (2, 6, R)** renvoie la liste : [(2 , 3 , 6) , (2 , 4 , 6)]
- La fonction **pc_chemins (7, 1, R)** renvoie la liste :
[(7 , 3 , 0 , 1) , (7 , 3 , 2 , 1) , (7 , 3 , 4 , 1) , (7 , 5 , 4 , 1) , (7 , 6 , 4 , 1)]
- La fonction **pc_chemins (1, 4, R)** renvoie la liste : [(1 , 4)]

~~~~~ **FIN DE L'ÉPREUVE** ~~~~~

**Épreuve d'Informatique – Session 2018 – Filière MP****Partie II : Bioinformatique : Recherche d'un motif**

*La bioinformatique, c'est quoi?*

Au cours des trente dernières années, la récolte de données en biologie a connu un boom quantitatif grâce notamment au développement de nouveaux moyens techniques servant à comprendre l'ADN et d'autres composants d'organismes vivants. Pour analyser ces données, plus nombreuses et plus complexes aussi, les scientifiques se sont tournés vers les nouvelles technologies de l'information. L'immense capacité de stockage et d'analyse des données qu'offre l'informatique leur a permis de gagner en puissance pour leurs recherches. La bioinformatique sert donc à stocker, traiter et analyser de grandes quantités de données de biologie. Le but est de mieux comprendre et mieux connaître les phénomènes et processus biologiques.

Dans le domaine de la bioinformatique, la recherche de sous-chaîne de caractères, appelée **motif**, dans un texte de taille très grande, était un composant important de beaucoup d'algorithmes. Puisqu'on travaille sur des textes de tailles très importantes, on a toujours cette obsession de l'efficacité des algorithmes : moins on a à faire de comparaisons, plus rapide sera l'exécution des algorithmes.

Le motif à rechercher, ainsi que le texte dans lequel on effectue la recherche, sont écrits dans un alphabet composé de quatre caractères : 'A', 'C', 'G' et 'T'. Ces caractères représentent les quatre bases de l'ADN : Adénine, Cytosine, Guanine et Thymine.

**Codage des caractères de l'alphabet**

L'alphabet est composé de quatre caractères seulement : 'A', 'C', 'G' et 'T'. Dans le but d'économiser la mémoire, on peut utiliser un codage binaire *réduit* pour ces quatre caractères.

**Q. 1 :** Quel est le *nombre minimum de bits* nécessaires pour coder quatre éléments ?

**Q. 2 :** Donner un exemple de code binaire pour chaque caractère de cet alphabet.

**Préfixe d'une chaîne de caractères**

Un *préfixe* d'une chaîne de caractères *S non vide*, est une sous-chaîne non vide de *S*, composée de la suite des caractères entre la première position de *S* et une certaine position dans *S*.

**Exemples :** *S* = 'TATCTAGCTA'

- ☐ La chaîne 'TAT' est un préfixe de 'TATCTAGCTA' ;
- ☐ La chaîne 'TATCTA' est un préfixe de 'TATCTAGCTA' ;
- ☐ La chaîne 'TATCTAGCTA' est un préfixe de 'TATCTAGCTA' ;
- ☐ La chaîne 'T' est un préfixe de 'TATCTAGCTA' ;
- ☐ La chaîne 'CTAGC' n'est pas un préfixe de *S*.

**Q. 3 :** Écrire une fonction **prefixe (M, S)**, qui reçoit en paramètres deux chaînes de caractères **M** et **S** non vides, et qui retourne **True** si la chaîne **M** est un préfixe de **S**, sinon, elle retourne **False**.

**Q. 4 :** Quel est le nombre de *comparaisons élémentaires* effectuées par la fonction **prefixe** (**M**, **S**), dans le pire cas ? Déduire la complexité de la fonction **prefixe** (**M**, **S**), dans le pire cas.

### Suffixe d'une chaîne de caractères

Un **suffixe** d'une chaîne de caractères **S** non vide, est une sous-chaîne non vide de **S**, composée de la suite des caractères, en partant d'une certaine position **p** dans **S**, jusqu'à la dernière position de **S**. L'entier **p** est appelé : **position du suffixe**.

**Exemples :** **S** = 'TATCTAGCTA'

- ☐ La chaîne 'TCTAGCTA' est un suffixe de 'TATCTAGCTA', sa position est : **2** ;
- ☐ La chaîne 'GCTA' est un suffixe de 'TATCTAGCTA', sa position est : **6** ;
- ☐ La chaîne 'TATCTAGCTA' est un suffixe de 'TATCTAGCTA', sa position est : **0** ;
- ☐ La chaîne 'A' est un suffixe de 'TATCTAGCTA', sa position est : **9** ;
- ☐ La chaîne 'CTAGC' n'est pas un suffixe de **S**.

**Q. 5 :** Écrire une fonction **liste\_suffixes** (**S**), qui reçoit en paramètre une chaîne de caractères **S** non vide, et qui renvoie une liste de tuples. Chaque tuple de cette liste est composé de deux éléments : un suffixe de **S**, et la position **p** de ce suffixe dans **S** ( $0 \leq p < \text{taille de } S$ ).

**Exemple :** **S** = 'TATCTAGCTA'

La fonction **liste\_suffixes** (**S**) renvoie la liste :

[('TATCTAGCTA', 0), ('ATCTAGCTA', 1), ('TCTAGCTA', 2), ('CTAGCTA', 3), ('TAGCTA', 4), ('AGCTA', 5), ('GCTA', 6), ('CTA', 7), ('TA', 8), ('A', 9)]

### Tri de la liste des suffixes

**Q. 6 :** Écrire une fonction **tri\_liste** (**L**), qui reçoit en paramètre la liste **L** des suffixes d'une chaîne de caractères non vide, créée selon le principe décrit dans la question 5. La fonction **tri\_liste**, trie les éléments de **L** dans l'ordre alphabétique des suffixes.

*NB : On ne doit pas utiliser la fonction **sorted()**, ou la méthode **sort()**.*

**Exemple :**

**L** = [('TATCTAGCTA', 0), ('ATCTAGCTA', 1), ('TCTAGCTA', 2), ('CTAGCTA', 3), ('TAGCTA', 4), ('AGCTA', 5), ('GCTA', 6), ('CTA', 7), ('TA', 8), ('A', 9)]

Après l'appel de la fonction **tri\_liste** (**L**), on obtient la liste suivante :

[('A', 9), ('AGCTA', 5), ('ATCTAGCTA', 1), ('CTA', 7), ('CTAGCTA', 3), ('GCTA', 6), ('TA', 8), ('TAGCTA', 4), ('TATCTAGCTA', 0), ('TCTAGCTA', 2)]

### Recherche dichotomique d'un motif :

La liste des suffixes **L** contient les tuples composés des suffixes d'une chaîne de caractères **S** non vide, et de leurs positions dans **S**. Si la liste **L** est triée dans l'ordre alphabétique des suffixes, alors, la recherche d'un motif **M** dans **S**, peut être réalisée par une simple *recherche dichotomique* dans la liste **L**.

**Q. 7 :** Écrire une fonction **recherche\_dichotomique** (**M**, **L**), qui utilise le *principe de la recherche dichotomique*, pour rechercher le motif **M** dans la liste des suffixes **L**, triée dans l'ordre alphabétique des suffixes : Si **M** est un préfixe d'un suffixe dans un tuple de **L**, alors la fonction retourne la position de ce suffixe. Si la fonction ne trouve pas le motif **M** recherché, alors la fonction retourne **None**.

**Exemple :**

La liste des suffixes de 'TATCTAGCTA', triée dans l'ordre alphabétique des suffixes est :

$L = [ ('A', 9), ('AGCTA', 5), ('ATCTAGCTA', 1), ('CTA', 7), ('CTAGCTA', 3), ('GCTA', 6), ('TA', 8), ('TAGCTA', 4), ('TATCTAGCTA', 0), ('TCTAGCTA', 2) ]$

- **recherche\_dichotomique** ('CTA', **L**) renvoie la position **3** ; 'CTA' est préfixe de 'CTAGCTA'
- **recherche\_dichotomique** ('TA', **L**) renvoie la position **0** ; 'TA' est préfixe de 'TATCTAGCTA'
- **recherche\_dichotomique** ('CAGT', **L**) renvoie **None**. 'CAGT' n'est préfixe d'aucun suffixe

**Q. 8 :** On pose **m** et **k** les tailles respectives de la chaîne **M** et la liste **L**. Déterminer, en fonction de **m** et **k**, la complexité de la fonction **recherche\_dichotomique** (**M**, **L**). Justifier votre réponse.

### L'arbre des suffixes d'une chaîne de caractères :

L'arbre des suffixes, d'une chaîne de caractères **S** non vide, est un **arbre n-aire** qui permet de représenter tous les suffixes de **S**. Cet arbre possède les propriétés suivantes :

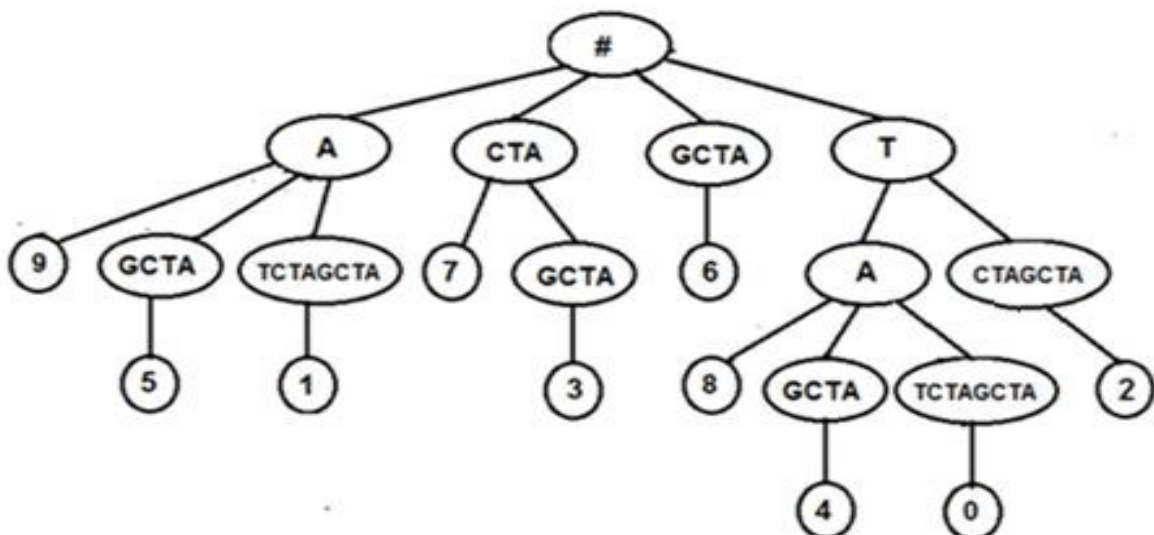
- La racine de l'arbre est marquée par le caractère '#' ;
- Chaque nœud de l'arbre contient une sous-chaîne de **S**, non vide ;
- Les premiers caractères, des sous-chaînes fils d'un même nœud, sont différents ;
- Chaque feuille de l'arbre contient un entier positif qui correspond à la position d'un suffixe dans la chaîne **S**.

**Exemple :** **S** = 'TATCTAGCTA'

La liste des suffixes de **S**, triée dans l'ordre alphabétique des suffixes est :

$L = [ ('A', 9), ('AGCTA', 5), ('ATCTAGCTA', 1), ('CTA', 7), ('CTAGCTA', 3), ('GCTA', 6), ('TA', 8), ('TAGCTA', 4), ('TATCTAGCTA', 0), ('TCTAGCTA', 2) ]$

L'arbre des suffixes est créé à partir de la liste des suffixes triée :



La racine de l'arbre '**#**' se trouve au niveau **0** de l'arbre, ses fils se trouvent au niveau **1**, ... . En parcourant une branche de l'arbre depuis le niveau **1**, et en effectuant la concaténation des sous-chaînes des nœuds, on obtient un suffixe de la chaîne **S** ; et dans la feuille, on trouve la position de ce suffixe dans **S**.

### Examples :

- En parcourant les nœuds de la première branche à droite de l'arbre '**T**' + '**CTAGCTA**', on obtient le suffixe '**TCTAGCTA**'. La feuille contient la position **2** de ce suffixe ;
- En parcourant les nœuds de la troisième branche à partir de la droite de l'arbre '**T**' + '**A**' + '**GCTA**', on obtient le suffixe '**TAGCTA**'. La feuille contient la position **4** de ce suffixe.

Pour représenter l'arbre des suffixes, on utilise une liste composée de deux éléments : Le nœud, et une liste contenant tous les fils de ce nœud :

- Un nœud **N** est représenté par la liste : [ **N**, [ **f<sub>1</sub>**, **f<sub>2</sub>**, ..., **f<sub>n</sub>** ] ;
- Chaque fils **f<sub>k</sub>** est représenté par la liste : [ **f<sub>k</sub>**, [ **tous les fils de f<sub>k</sub>** ] ] ;
- Une feuille **F** est représentée par la liste : [ **F**, [ ] ].

On suppose que la fonction **arbre\_suffixes** (**S**), reçoit en paramètre une chaîne de caractères **S** non vide, et construit et renvoie l'arbre des suffixes de **S**, représenté par une liste, selon le principe décrit ci-dessus.

### Example :

**arbre\_suffixes** ('TATCTAGCTA') renvoie la liste **R** suivante :

```
R=[ '#',[ ['A',[ [9, []],[ 'GCTA',[ [5,[[]]]],[ 'TCTAGCTA',[ [1,[[]]]]]],[ 'CTA',[ [7,[[]],[ 'GCTA',[ [3,[[]]]]] ]],[ 'GCTA',[ [6,[[]]]],[ 'T',[ [ 'A',[ [8,[[]],[ 'GCTA',[ [4,[[]]] ]],[ 'TCTAGCTA',[ [0,[[]]]]]],[ 'CTAGCTA',[ [2,[[]]]]]]]]
```

**Q. 9 :** Écrire une fonction **feuilles (R)**, qui reçoit en paramètre une liste **R** représentant l'arbre des suffixes d'une chaîne de caractères non vide, et qui retourne la liste contenant toutes les feuilles de l'arbre **R**.

**Exemple : feuilles (R)** renvoie la liste [ 9, 5, 1, 7, 3, 6, 8, 4, 0, 2 ]

L'intérêt, de l'arbre des suffixes, c'est qu'il permet d'effectuer une recherche de **toutes les occurrences** d'un motif dans **un temps qui dépend uniquement de la longueur du motif**, non de la chaîne dans laquelle on effectue la recherche. En effet, la recherche d'un motif se réalise par la lecture du motif recherché et par le parcours simultané de l'arbre des suffixes. Trois cas peuvent survenir :

- **On se retrouve sur un nœud , et le motif est un préfixe de ce nœud :** Dans ce cas, les feuilles du sous arbre de ce nœud, correspondent aux positions de toutes les occurrences du motif dans la chaîne.
- **On se retrouve sur un nœud, et ce nœud est un préfixe du motif :** Dans ce cas, on continue la recherche parmi les fils de ce nœud.
- **On ne peut plus descendre dans l'arbre :** Dans ce cas, il n'y a pas d'occurrence du motif recherché dans la chaîne.

**Q. 10 :** Écrire une fonction **recherche\_arbre** (**M**, **R**), qui reçoit en paramètres un motif **M** non vide, et une liste **R** représentant l'arbre des suffixes d'une chaîne de caractères non vide. La fonction retourne une liste contenant les positions de toutes les occurrences de **M** dans cette chaîne de caractères.

**Exemples :**

**R** est la liste qui représente l'arbre des suffixes de la chaîne **S** = 'TATCTAGCTA'.

- **recherche\_arbre** ('TA', **R**) renvoie la liste : [ 8 , 4 , 0 ]
- **recherche\_arbre** ('TCTAG', **R**) renvoie la liste : [ 2 ]
- **recherche\_arbre** ('CTA', **R**) renvoie la liste : [ 7 , 3 ]
- **recherche\_arbre** ('TACG', **R**) renvoie la liste : [ ]

**Plus long motif commun**

Le problème du **plus long motif commun** à deux chaînes de caractères **P** et **Q** non vides, consiste à déterminer le (ou les) plus long motif qui est en même temps sous-chaîne de **P** et de **Q**. Ce problème se généralise à la recherche du plus long motif commun à plusieurs chaînes de caractères.

Pour rechercher le plus long motif commun à deux chaînes de caractères **P** et **Q** non vides, on commence par créer une matrice **M** remplie par des zéros : Le nombre de lignes de **M** est égal à la taille de **P**, et le nombre de colonnes de **M** est égal à la taille de **Q**. Ensuite, on compare les caractères de **P** avec ceux de **Q**, et on écrit le nombre de caractères successifs identiques, dans la case correspondante de la matrice **M**.

**Exemple :** **P** = 'GCTAGCATT' et **Q** = 'CATTGTAGCT'

La matrice **M**, de comparaison des caractères de **P** avec ceux de **Q**, est la suivante :

|   | C | A | T | T | G | T | A | G | C | T |
|---|---|---|---|---|---|---|---|---|---|---|
| G | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| C | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 0 |
| T | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 0 | 3 |
| A | 0 | 1 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 0 |
| G | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 3 | 0 | 0 |
| C | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 0 |
| A | 0 | 2 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| T | 0 | 0 | 3 | 1 | 0 | 1 | 0 | 0 | 0 | 1 |
| T | 0 | 0 | 1 | 4 | 0 | 1 | 0 | 0 | 0 | 1 |

À partir de cette matrice **M**, On peut extraire les plus longs motifs communs à **P** et **Q** : 'CATT' et 'TAGC'

**Q-18 :** Écrire une fonction : **matrice (P, Q)**, qui reçoit en paramètres deux chaînes de caractères **P** et **Q** non vides, et qui retourne la matrice **M** de comparaison des caractères de **P** avec ceux de **Q**, selon le principe décrit dans l'exemple ci-dessus.

**Exemple :** **P** = 'GCTAGCATT' et **Q** = 'CATTGTAGCT'

**matrice (P, Q)** renvoie la matrice **M** suivante :

[ [0, 0, 0, 0, 1, 0, 0, 1, 0, 0], [1, 0, 0, 0, 0, 0, 0, 0, 2, 0], [0, 0, 1, 1, 0, 1, 0, 0, 0, 3], [...], ... ]



**Q-19** : Écrire une fonction **plus\_long\_mc (P, M)**, qui reçoit en paramètres une chaîne de caractères **P** non vide, et la matrice **M** de comparaison des caractères de la chaîne **P** avec ceux d'une autre chaîne **Q** non vide aussi, selon le principe décrit dans l'exemple ci-dessus. La fonction renvoie la liste, sans doublons, de tous les plus longs motifs communs à **P** et **Q**.

**Exemple :**

**P** = 'GCTAGCATT' et **M** est la matrice de comparaison des caractères de **P** avec ceux de la chaîne 'CATTGTAGCT'.

**plus\_long\_mc (P, M)** retourne la liste des plus longs motifs communs : [ 'TAGC' , 'CATT' ]

**Plus long motif commun dans un fichier**

On considère le fichier texte dont le chemin absolu est : 'C:\génomés.txt'. On suppose que ce fichier contient au moins deux lignes.

**Q-20** : Écrire le code python qui permet d'afficher les plus longs motifs communs, à toutes les lignes de ce fichier texte, sans doublons.

**Exemple :**

On suppose que le fichier 'C:\génomés.txt' est composé des lignes suivantes :

```
GATTCGAGGCTAAACTAGCTAA
GATTCGAGGCTCTAGCTATTCGAGGG
CTAGCTATAGGCTAGCTACCATTATTCGAG
CGCTAGCTACATTCGAGGCTAAAC
ATTCGAGAGGGCTAACTAGCTAAGCCA
```

Le programme affiche les plus longs motifs communs à toutes les lignes de ce fichier :

'ATTCGAG' , 'CTAGCTA'

~~~~~ **FIN** ~~~~~

Épreuve d'Informatique – Session 2018 – Filière PSI

Bioinformatique : Recherche d'un motif

La bioinformatique, c'est quoi?

Au cours des trente dernières années, la récolte de données en biologie a connu un boom quantitatif grâce notamment au développement de nouveaux moyens techniques servant à comprendre l'ADN et d'autres composants d'organismes vivants. Pour analyser ces données, plus nombreuses et plus complexes aussi, les scientifiques se sont tournés vers les nouvelles technologies de l'information. L'immense capacité de stockage et d'analyse des données qu'offre l'informatique leur a permis de gagner en puissance pour leurs recherches. La bioinformatique sert donc à stocker, traiter et analyser de grandes quantités de données de biologie. Le but est de mieux comprendre et mieux connaître les phénomènes et processus biologiques.

Dans le domaine de la bioinformatique, la recherche de sous-chaîne de caractères, appelée **motif**, dans un texte de taille très grande, était un composant important de beaucoup d'algorithmes. Puisqu'on travaille sur des textes de tailles très importantes, on a toujours cette obsession de l'efficacité des algorithmes : moins on a à faire de comparaisons, plus rapide sera l'exécution des algorithmes.

Le motif à rechercher, ainsi que le texte dans lequel on effectue la recherche, sont écrits dans un alphabet composé de quatre caractères : 'A', 'C', 'G' et 'T'. Ces caractères représentent les quatre bases de l'ADN : Adénine, Cytosine, Guanine et Thymine.

Codage des caractères de l'alphabet

L'alphabet est composé de quatre caractères seulement : 'A', 'C', 'G' et 'T'. Dans le but d'économiser la mémoire, on peut utiliser un codage binaire *réduit* pour ces quatre caractères.

Q. 1 : Quel est le nombre minimum de bits nécessaires pour coder quatre éléments ?

Q. 2 : Donner un exemple de code binaire pour chaque caractère de cet alphabet.

Préfixe d'une chaîne de caractères

Un **préfixe** d'une chaîne de caractères *S* non vide, est une sous-chaîne non vide de *S*, composée de la suite des caractères entre la première position de *S* et une certaine position dans *S*.

Exemples : *S* = 'TATCTAGCTA'

La chaîne 'TAT' est un préfixe de 'TATCTAGCTA' ;

La chaîne 'TATCTA' est un préfixe de 'TATCTAGCTA' ;

La chaîne 'TATCTAGCTA' est un préfixe de 'TATCTAGCTA' ;

La chaîne 'T' est un préfixe de 'TATCTAGCTA' ;

La chaîne 'CTAGC' n'est pas un préfixe de *S*.

Q. 3 : Écrire une fonction **prefixe (M, S)**, qui reçoit en paramètres deux chaînes de caractères **M** et **S** non vides, et qui retourne **True** si la chaîne **M** est un préfixe de **S**, sinon, elle retourne **False**.

Q. 4 : Quel est le nombre de *comparaisons élémentaires* effectuées par la fonction **prefixe** (**M**, **S**), dans le pire cas ? Déduire la complexité de la fonction **prefixe** (**M**, **S**), dans le pire cas.

Suffixe d'une chaîne de caractères

Un **suffixe** d'une chaîne de caractères **S** non vide, est une sous-chaîne non vide de **S**, composée de la suite des caractères, en partant d'une certaine position **p** dans **S**, jusqu'à la dernière position de **S**. L'entier **p** est appelé : **position du suffixe**.

Exemples : **S** = 'TATCTAGCTA'

La chaîne 'TCTAGCTA' est un suffixe de 'TATCTAGCTA', sa position est : **2** ;

La chaîne 'GCTA' est un suffixe de 'TATCTAGCTA', sa position est : **6** ;

La chaîne 'TATCTAGCTA' est un suffixe de 'TATCTAGCTA', sa position est : **0** ;

La chaîne 'A' est un suffixe de 'TATCTAGCTA', sa position est : **9** ;

La chaîne 'CTAGC' n'est pas un suffixe de **S**.

Q. 5 : Écrire une fonction **liste_suffixes** (**S**), qui reçoit en paramètre une chaîne de caractères **S** non vide, et qui renvoie une liste de tuples. Chaque tuple de cette liste est composé de deux éléments : un suffixe de **S**, et la position **p** de ce suffixe dans **S** ($0 \leq p < \text{taille de } S$). Les tuples de cette liste sont ordonnés dans l'ordre croissant des positions des suffixes.

Exemple : **S** = 'TATCTAGCTA'

La fonction **liste_suffixes** (**S**) renvoie la liste :

[('TATCTAGCTA', 0), ('ATCTAGCTA', 1), ('TCTAGCTA', 2), ('CTAGCTA', 3), ('TAGCTA', 4), ('AGCTA', 5), ('GCTA', 6), ('CTA', 7), ('TA', 8), ('A', 9)]

Recherche séquentielle d'un motif

La **recherche séquentielle**, d'un motif **M** dans une chaîne de caractères **S** non vide, consiste à parcourir la liste **L** des suffixes de **S** (voir Question. 5), en cherchant si le motif **M** est un préfixe du suffixe d'un tuple de la liste **L**. Si oui, la fonction retourne la position de ce suffixe.

*Le parcours, de la liste **L**, ne doit considérer que les positions possibles pour le motif **M**, c'est-à-dire les positions **p** tels que : $0 \leq p \leq (\text{taille de } L - \text{taille de } M)$.*

Q. 6 : Écrire une fonction : **recherche_sequentielle** (**M**, **L**), qui, en utilisant le principe de la *recherche séquentielle* dans **L**, renvoie la position du premier tuple, tel que **M** est un préfixe du suffixe de ce tuple, s'il existe ; sinon, la fonction renvoie **None**.

NB : Dans cette fonction, on ne doit pas utiliser la méthode `index()`.

Exemples : La liste des suffixes de la chaîne 'TATCTAGCTA' est :

L = [('TATCTAGCTA', 0), ('ATCTAGCTA', 1), ('TCTAGCTA', 2), ('CTAGCTA', 3), ('TAGCTA', 4), ('AGCTA', 5), ('GCTA', 6), ('CTA', 7), ('TA', 8), ('A', 9)]

recherche_sequentielle ('CTA', **L**) renvoie la position **3** ; 'CTA' est préfixe de 'CTAGCTA'

recherche_sequentielle ('TA', **L**) renvoie la position **0** ; 'TA' est préfixe de 'TATCTAGCTA'

recherche_sequentielle ('CAGTA', **L**) renvoie **None**. 'CAGT' n'est préfixe d'aucun suffixe

Q. 7 : On pose **m** et **k** les tailles respectives de la chaîne **M** et la liste **L**. Déterminer, en fonction de **m** et **k**, la complexité de la fonction **recherche_sequentielle** (**M**, **L**), dans le pire cas. Et, donner un exemple, de chaînes **M** et **S**, qui atteigne cette complexité.

Tri de la liste des suffixes

Q. 8 : Écrire une fonction **tri_liste** (**L**), qui reçoit en paramètre la liste **L** des suffixes d'une chaîne de caractères non vide, créée selon le principe décrit dans la question 5. La fonction **tri_liste**, trie les éléments de **L** dans l'ordre alphabétique des suffixes.

*NB : On ne doit pas utiliser la fonction **sorted()**, ou la méthode **sort()**.*

Exemple :

L = [('TATCTAGCTA', 0), ('ATCTAGCTA', 1), ('TCTAGCTA', 2), ('CTAGCTA', 3), ('TAGCTA', 4), ('AGCTA', 5), ('GCTA', 6), ('CTA', 7), ('TA', 8), ('A', 9)]

Après l'appel de la fonction **tri_liste** (**L**), on obtient la liste suivante :

[('A', 9), ('AGCTA', 5), ('ATCTAGCTA', 1), ('CTA', 7), ('CTAGCTA', 3), ('GCTA', 6), ('TA', 8), ('TAGCTA', 4), ('TATCTAGCTA', 0), ('TCTAGCTA', 2)]

Recherche dichotomique d'un motif :

La liste des suffixes **L** contient les tuples composés des suffixes d'une chaîne de caractères **S** non vide, et de leurs positions dans **S**. Si la liste **L** est triée dans l'ordre alphabétique des suffixes, alors, la recherche d'un motif **M** dans **S**, peut être réalisée par une simple *recherche dichotomique* dans la liste **L**.

Q. 9 : Écrire une fonction **recherche_dichotomique** (**M**, **L**), qui utilise le *principe de la recherche dichotomique*, pour rechercher le motif **M** dans la liste des suffixes **L** triée dans l'ordre alphabétique des suffixes : Si **M** est un préfixe d'un suffixe dans un tuple de **L**, alors la fonction retourne la position de ce suffixe. Si la fonction ne trouve pas le motif **M** recherché, alors la fonction retourne **None**.

Exemple :

La liste des suffixes de 'TATCTAGCTA', triée dans l'ordre alphabétique des suffixes est :

L = [('A', 9), ('AGCTA', 5), ('ATCTAGCTA', 1), ('CTA', 7), ('CTAGCTA', 3), ('GCTA', 6), ('TA', 8), ('TAGCTA', 4), ('TATCTAGCTA', 0), ('TCTAGCTA', 2)]

recherche_dichotomique ('CTA', **L**) renvoie la position **3** ; 'CTA' est préfixe de 'CTAGCTA'
recherche_dichotomique ('TA', **L**) renvoie la position **0** ; 'TA' est préfixe de 'TATCTAGCTA'
recherche_dichotomique ('CAGT', **L**) renvoie **None**. 'CAGT' n'est préfixe d'aucun suffixe

Q. 10 : On pose **m** et **k** les tailles respectives de la chaîne **M** et la liste **L**. Déterminer, en fonction de **m** et **k**, la complexité de la fonction **recherche_dichotomique** (**M**, **L**), dans le pire cas. Justifier votre réponse.

L'arbre des suffixes d'une chaîne de caractères :

L'arbre des suffixes, d'une chaîne de caractères **S** non vide, est un **arbre n-aire** qui permet de représenter tous les suffixes de **S**. Cet arbre possède les propriétés suivantes :

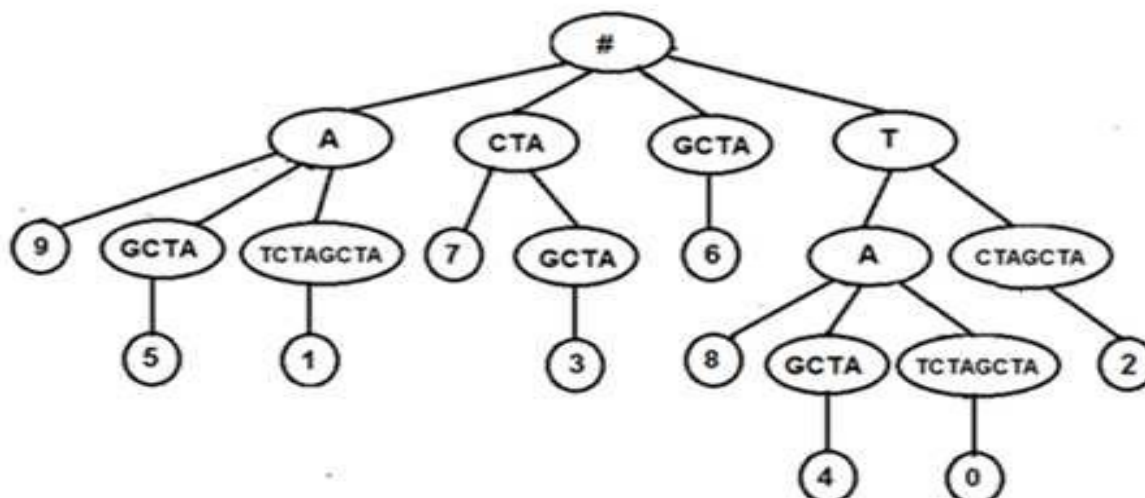
- La racine de l'arbre est marquée par le caractère **#** ;
- Chaque nœud de l'arbre contient une sous-chaîne de **S**, non vide ;
- Les premiers caractères, des sous-chaînes fils d'un même nœud, sont différents ;
- Chaque feuille de l'arbre contient un entier positif qui correspond à la position d'un suffixe dans la chaîne **S**.

Exemple : **S** = 'TATCTAGCTA'

La liste des suffixes de **S**, triée dans l'ordre alphabétique des suffixes est :

L = [('A', 9) , ('AGCTA', 5) , ('ATCTAGCTA', 1) , ('CTA', 7) , ('CTAGCTA', 3) , ('GCTA', 6) , ('TA', 8) , ('TAGCTA', 4) , ('TATCTAGCTA', 0) , ('TCTAGCTA', 2)]

L'arbre des suffixes est créé à partir de la liste des suffixes triée :



La racine de l'arbre **#** se trouve au niveau **0** de l'arbre, ses fils se trouvent au niveau **1**, En parcourant une branche de l'arbre depuis le niveau **1**, et en effectuant la concaténation des sous-chaînes des nœuds, on obtient un suffixe de la chaîne **S** ; et dans la feuille, on trouve la position de ce suffixe dans **S**.

Exemples :

En parcourant les nœuds de la première branche à droite de l'arbre **'T' + 'CTAGCTA'**, on obtient le suffixe **'TCTAGCTA'**. La feuille contient la position **2** de ce suffixe ;

En parcourant les nœuds de la troisième branche à partir de la droite de l'arbre **'T' + 'A' + 'GCTA'**, on obtient le suffixe **'TAGCTA'**. La feuille contient la position **4** de ce suffixe.

Pour représenter l'arbre des suffixes, on utilise une liste composée de deux éléments : Le nœud, et une liste contenant tous les fils de ce nœud :

Un nœud **N** est représenté par la liste : [**N** , [**f₁** , **f₂** , ... , **f_n**] ;

Chaque fils **f_k** est représenté par la liste : [**f_k** , [**tous les fils de f_k**]] ;

Une feuille **F** est représentée par la liste : [**F** , []].

Épreuve d'Informatique – Session 2018 – Filière TSI

Bioinformatique : Recherche d'un motif

La bioinformatique, c'est quoi?

Au cours des trente dernières années, la récolte de données en biologie a connu un boom quantitatif grâce notamment au développement de nouveaux moyens techniques servant à comprendre l'ADN et d'autres composants d'organismes vivants. Pour analyser ces données, plus nombreuses et plus complexes aussi, les scientifiques se sont tournés vers les nouvelles technologies de l'information. L'immense capacité de stockage et d'analyse des données qu'offre l'informatique leur a permis de gagner en puissance pour leurs recherches. La bioinformatique sert donc à stocker, traiter et analyser de grandes quantités de données de biologie. Le but est de mieux comprendre et mieux connaître les phénomènes et processus biologiques.

Dans le domaine de la bioinformatique, la recherche de sous-chaîne de caractères, appelée **motif**, dans un texte de taille importante, était un composant important dans beaucoup d'algorithmes. Puisqu'on travaille sur des textes de tailles très grandes, on a toujours cette obsession de l'efficacité des algorithmes : moins on a à faire de comparaisons, plus rapide sera l'exécution des algorithmes.

Le motif à rechercher, ainsi que le texte dans lequel on effectue la recherche, sont écrits dans un alphabet composé de quatre caractères : 'A', 'C', 'G' et 'T'. Ces caractères représentent les quatre bases de l'ADN : Adénine, Cytosine, Guanine et Thymine.

Codage des caractères de l'alphabet

L'alphabet est composé de quatre caractères seulement : 'A', 'C', 'G' et 'T'. Dans le but d'économiser la mémoire, on peut utiliser un codage binaire réduit pour ces quatre caractères.

Q. 1 : Quel est le nombre minimum de bits nécessaires pour coder quatre éléments ?

Q. 2 : Donner un exemple de code binaire pour chaque caractère de cet alphabet.

Préfixe d'une chaîne de caractères

*Un **préfixe** d'une chaîne de caractères S non vide, est une sous-chaîne non vide de S , composée de la suite des caractères entre la première position de S et une certaine position dans S .*

Exemples : $S = \text{'TATCTAGCTA'}$

- ☐ La chaîne 'TAT' est un préfixe de 'TATCTAGCTA' ;
- ☐ La chaîne 'TATCTA' est un préfixe de 'TATCTAGCTA' ;
- ☐ La chaîne 'TATCTAGCTA' est un préfixe de 'TATCTAGCTA' ;
- ☐ La chaîne 'T' est un préfixe de 'TATCTAGCTA' ;
- ☐ La chaîne 'CTAGC' n'est pas un préfixe de S .

Q. 3 : Écrire une fonction **prefixe** (M , S), qui reçoit en paramètres deux chaînes de caractères M et S non vides, et qui retourne **True** si la chaîne M est un préfixe de S , sinon, elle retourne **False**.

Q. 4 : Quel est le nombre de *comparaisons élémentaires* effectuées par la fonction **prefixe** (**M**, **S**), dans le pire cas ? Déduire la complexité de la fonction **prefixe** (**M**, **S**) dans le pire cas.

Suffixe d'une chaîne de caractères

Un **suffixe** d'une chaîne de caractères **S** non vide, est une sous-chaîne non vide de **S**, composée de la suite des caractères, en partant d'une certaine position **p** dans **S**, jusqu'à la dernière position de **S**. L'entier **p** est appelé : position du suffixe.

Exemples : **S** = 'TATCTAGCTA'

- ☐ La chaîne 'TCTAGCTA' est un suffixe de 'TATCTAGCTA', sa position est : **2** ;
- ☐ La chaîne 'GCTA' est un suffixe de 'TATCTAGCTA', sa position est : **6** ;
- ☐ La chaîne 'TATCTAGCTA' est un suffixe de 'TATCTAGCTA', sa position est : **0** ;
- ☐ La chaîne 'A' est un suffixe de 'TATCTAGCTA', sa position est : **9** ;
- ☐ La chaîne 'CTAGC' n'est pas un suffixe de **S**.

Q. 5 : Écrire une fonction **liste_suffixes** (**S**), qui reçoit en paramètre une chaîne de caractères **S** non vide, et qui renvoie une liste de tuples. Chaque tuple de cette liste est composé de deux éléments : un suffixe de **S**, et la position **p** de ce suffixe dans **S** ($0 \leq p < \text{taille de } S$). Les tuples de cette liste sont ordonnés dans l'ordre croissant des positions des suffixes.

Exemple : **S** = 'TATCTAGCTA'

La fonction **liste_suffixes** (**S**) renvoie la liste :

[('TATCTAGCTA', 0), ('ATCTAGCTA', 1), ('TCTAGCTA', 2), ('CTAGCTA', 3), ('TAGCTA', 4), ('AGCTA', 5), ('GCTA', 6), ('CTA', 7), ('TA', 8), ('A', 9)]

Recherche séquentielle d'un motif

La **recherche séquentielle**, d'un motif **M** dans une chaîne de caractères **S** non vide, consiste à parcourir la liste **L** des suffixes de **S** (voir Question. 5), en cherchant si le motif **M** est un préfixe du suffixe d'un tuple de la liste **L**. Si oui, la fonction retourne la position de ce suffixe.

*Le parcours, de la liste **L**, ne doit considérer que les positions possibles pour le motif **M**, c'est-à-dire les positions **p** tels que : $0 \leq p \leq (\text{taille de } L - \text{taille de } M)$.*

Q. 6 : Écrire une fonction **recherche_sequentielle** (**M**, **L**), qui, en utilisant le principe de la *recherche séquentielle* dans la liste des suffixes **L**, renvoie la position du premier tuple, tel que **M** est un préfixe du suffixe de ce tuple, s'il existe ; sinon, la fonction renvoie **None**.

NB : Dans cette fonction, on ne doit pas utiliser la méthode `index()`.

Exemples : La liste des suffixes de la chaîne 'TATCTAGCTA' est :

L = [('TATCTAGCTA', 0), ('ATCTAGCTA', 1), ('TCTAGCTA', 2), ('CTAGCTA', 3), ('TAGCTA', 4), ('AGCTA', 5), ('GCTA', 6), ('CTA', 7), ('TA', 8), ('A', 9)]

- ☐ **recherche_sequentielle** ('CTA', **L**) renvoie la position **3** ; 'CTA' est préfixe de 'CTAGCTA'
- ☐ **recherche_sequentielle** ('TA', **L**) renvoie la position **0** ; 'TA' est préfixe de 'TATCTAGCTA'
- ☐ **recherche_sequentielle** ('CAGTA', **L**) renvoie **None**. 'CAGT' n'est préfixe d'aucun suffixe

Q. 7 : On pose **m** et **k** les tailles respectives de la chaîne **M** et la liste **L**. Déterminer, en fonction de **m** et **k**, la complexité de la fonction recherche_sequentielle (M,L), dans le pire cas. Et, donner un exemple, de chaînes **M** et **S**, qui atteigne cette complexité.

Tri de la liste des suffixes

Q. 8 : Écrire une fonction **tri_liste** (**L**), qui reçoit en paramètre la liste **L** des suffixes d'une chaîne de caractères non vide, créée selon le principe décrit dans la question 5. La fonction **tri_liste**, trie les éléments de **L** dans l'ordre alphabétique des suffixes.

NB : On ne doit pas utiliser la fonction `sorted()`, ou la méthode `sort()`.

Exemple :

$L = [(\text{'TATCTAGCTA'}, 0), (\text{'ATCTAGCTA'}, 1), (\text{'TCTAGCTA'}, 2), (\text{'CTAGCTA'}, 3), (\text{'TAGCTA'}, 4), (\text{'AGCTA'}, 5), (\text{'GCTA'}, 6), (\text{'CTA'}, 7), (\text{'TA'}, 8), (\text{'A'}, 9)]$

Après l'appel de la fonction **tri_liste** (**L**), on obtient la liste suivante :

[('A', 9), ('AGCTA', 5), ('ATCTAGCTA', 1), ('CTA', 7), ('CTAGCTA', 3), ('GCTA', 6), ('TA', 8), ('TAGCTA', 4), ('TATCTAGCTA', 0), ('TCTAGCTA', 2)]

Recherche dichotomique d'un motif :

La liste des suffixes **L** contient les tuples composés des suffixes d'une chaîne de caractères **S** non vide, et de leurs positions dans **S**. Si la liste **L** est triée dans l'ordre alphabétique des suffixes, alors, la recherche d'un motif **M** dans **S**, peut être réalisée par une simple *recherche dichotomique* dans la liste **L**.

Q. 9 : Écrire une fonction **recherche_dichotomique** (**M**, **L**), qui utilise le *principe de la recherche dichotomique*, pour rechercher le motif **M** dans la liste des suffixes **L** triée dans l'ordre alphabétique des suffixes : Si **M** est un préfixe d'un suffixe dans un tuple de **L**, alors la fonction retourne la position de ce suffixe. Si la fonction ne trouve pas le motif **M** recherché, alors la fonction retourne **None**.

Example :

La liste des suffixes de 'TATCTAGCTA', triée dans l'ordre alphabétique des suffixes est :

$L = [('A', 9), ('AGCTA', 5), ('ATCTAGCTA', 1), ('CTA', 7), ('CTAGCTA', 3), ('GCTA', 6), ('TA', 8), ('TAGCTA', 4), ('TATCTAGCTA', 0), ('TCTAGCTA', 2)]$

- ❑ **recherche_dichotomique** ('CTA', L) renvoie la position **3** ; *'CTA' est préfixe de 'CTAGCTA'*
- ❑ **recherche_dichotomique** ('TA', L) renvoie la position **0** ; *'TA' est un préfixe de 'TATCTAGCTA'*
- ❑ **recherche_dichotomique** ('CAGT', L) renvoie **None**. *'CAGT' n'est préfixe d'aucun suffixe*

Q. 10 : On pose **m** et **k** les tailles respectives de la chaîne **M** et la liste **L**. Déterminer, en fonction de **m** et **k**, la complexité de la fonction **recherche_dichotomique** (**M**, **L**), dans le pire cas.

Justifier votre réponse.

[illegible]

Épreuve d'Informatique – Session 2017 – Filière MP/ PSI/ TSI

Partie II : Programmation Python

La compression de HUFFMAN

La **compression de Huffman** est une méthode de compression de données sans perte proposé par « David Huffman » en 1952. C'est un processus qui permet de compresser des données informatiques afin de libérer de la place dans la mémoire d'un ordinateur. Or tout fichier informatique (qu'il s'agisse d'un fichier texte, d'une image ou d'une musique ...) est formé d'une suite de caractères. Chacun de ces caractères étant lui-même codé par une suite de 0 et de 1. L'idée du codage de Huffman est de repérer les caractères les plus fréquents et de leur attribuer des codes courts (c'est-à-dire nécessitant moins de 0 et de 1) alors que les caractères les moins fréquents auront des codes longs. Pour déterminer le code de chaque caractère on utilise un arbre binaire. Cet arbre est également utilisé pour le décodage.

Dans cette partie, on s'intéressera à appliquer la compression de Huffman pour compresser une liste de nombres entiers, contenant des éléments en répétition.

Exemple : $L = [12, 29, 55, 29, 31, 8, 12, 46, 29, 8, 12, 29, 31, 29, 8, 29, 8]$

NB : Dans toute cette partie, on suppose que la liste L contient au moins deux éléments différents.

II. 1- L'espace mémoire occupé par un entier positif

Quel est le plus grand nombre entier positif, avec bit de signe, qu'on peut coder sur 12 bits ? Justifier votre réponse.

II. 2- Écriture binaire d'un nombre entier

II. 2- a) Écrire la fonction : **binaire(N)**, qui reçoit en paramètre un entier naturel N , et qui retourne une chaîne de caractères qui contient la représentation binaire de N . *Le bit du poids le plus fort se trouve à la fin de la chaîne. Ne pas utiliser la fonction prédéfinie `bin()` de Python.*

Exemple : **binaire(23)** retourne la chaîne : "11101"

II. 2- b) Déterminer la complexité de la fonction **binaire(N)**.

II. 2- c) La méthode dite de **Horner** (*William George Horner : 1786-1837*) est une méthode très pratique, utilisée pour améliorer le calcul de la valeur d'un polynôme, en réduisant le nombre de multiplications.

Soit le polynôme P à coefficients réels suivant :

$$P(x) = a_n * x^n + a_{n-1} * x^{n-1} + \dots + a_2 * x^2 + a_1 * x + a_0$$

Pour calculer $P(k)$, la méthode de Horner effectue le calcul comme suit :

$$P(k) = ((\dots ((a_n * k + a_{n-1}) * k + a_{n-2}) * k + \dots + a_2) * k + a_1) * k + a_0$$

En utilisant le principe de la méthode de Horner, écrire la fonction de complexité linéaire : **entier(B)**, qui reçoit en paramètre une chaîne de caractères B contenant la représentation binaire d'un entier naturel, dont le premier bit est celui du poids le plus faible, et qui calcule et retourne la valeur décimale de cet entier.

Exemple : **entier("11101")** retourne le nombre entier : **23** ($23 = 1*2^0 + 1*2^1 + 1*2^2 + 0*2^3 + 1*2^4$)

II. 3- Liste des fréquences :

La première étape de la méthode de compression de Huffman consiste à compter le nombre d'occurrences de chaque élément de la liste des entiers.

Écrire une fonction : **frequences (L)**, qui reçoit en paramètre une liste d'entiers **L**, et qui retourne une liste de tuples : Chaque tuple est composé d'un élément de **L** et de son occurrence (répétition) dans **L**. Les premiers éléments des tuples de la liste des fréquences doivent être tous différents (sans doublon).

Exemple : **L** = [12, 29, 46, 29, 31, 8, 12, 55, 29, 8, 12, 29, 31, 29, 8, 29, 8]

La fonction **frequences (L)** retourne la liste **F** = [(8, 4), (12, 3), (46, 1), (55, 1), (29, 6), (31, 2)]

II. 4- Tri de la liste des fréquences :

La liste **F** des fréquences, des différents éléments de la liste des entiers, étant créée. L'étape suivante est de trier cette liste **F** dans l'ordre croissant des occurrences (les 2^{èmes} éléments des tuples).

Écrire la fonction: **tri (F)**, qui reçoit en paramètre la liste **F** des fréquences, et qui trie les éléments de la liste **F** dans l'ordre croissant des occurrences. *Cette fonction doit être de complexité quasi-linéaire $O(n\log(n))$. Ne pas utiliser ni la fonction `sorted()`, ni la méthode `sort()` de Python.*

Exemple : **F** = [(12, 3), (29, 6), (55, 1), (31, 2), (8, 4), (46, 1)]

La fonction **tri (F)** retourne la liste : [(46, 1), (55, 1), (31, 2), (12, 3), (8, 4), (29, 6)]

II. 5- Création de l'arbre binaire de HUFFMAN :

Pour représenter un arbre binaire, on va utiliser une liste composée de trois éléments :

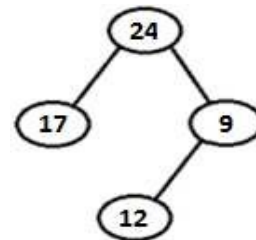
Chaque nœud est représenté par la liste : [Père , [Fils gauche] , [Fils droit]]

Chaque feuille est représentée par la liste : [Feuille , [] , []]

Exemple :

L'arbre binaire suivant, sera représenté par la liste :

A = [24, [17, [] , []], [9, [12, [] , []], []]]



L'arbre binaire de Huffman est créé à partir de la liste des fréquences, selon le procédé suivant :

Trier la liste des fréquences dans l'ordre croissant des occurrences ;

Tant que la liste des fréquences contient au moins deux éléments, on répète les opérations :

- *Retirer les deux premiers éléments dont les poids (2^{ème} élément de chaque tuple) sont les plus faibles, et créer un tuple dont le poids est la somme des poids de ces deux éléments ;*
- *Insérer le tuple obtenu dans la liste des fréquences, de façon à ce qu'elle reste croissante.*

Les feuilles de l'arbre binaire obtenu sont les différents éléments de la liste à compresser.

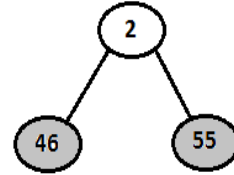
Exemple :

La liste F des fréquences suivante, est triée dans l'ordre croissant des occurrences.

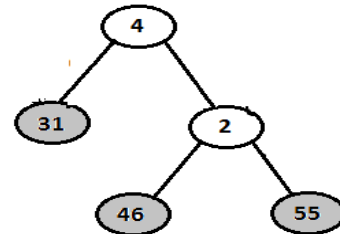
$F = [(46, 1), (55, 1), (31, 2), (12, 3), (8, 4), (29, 6)]$

1^{ère} étape

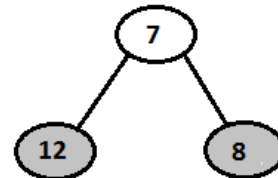
$F = [(46, 1), (55, 1), (31, 2), (12, 3), (8, 4), (29, 6)]$
 Retirer : **(46, 1)** et **(55, 1)**
 Insérer le tuple **(A, 2)** dans la liste F
 La liste F triée devient :
 $F = [(31, 2), (A, 2), (12, 3), (8, 4), (29, 6)]$

A**2^{ème} étape**

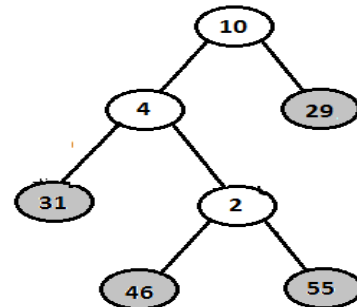
$F = [(31, 2), (A, 2), (12, 3), (8, 4), (29, 6)]$
 Retirer : **(31, 2)** et **(A, 2)**
 Insérer le tuple **(A, 4)** dans la liste F
 La liste F triée devient :
 $F = [(12, 3), (8, 4), (A, 4), (29, 6)]$

A**3^{ème} étape**

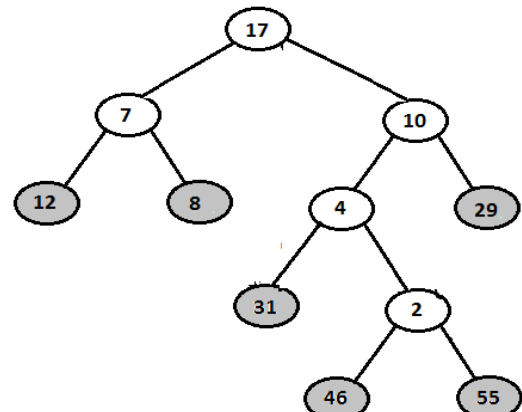
$F = [(12, 3), (8, 4), (A, 4), (29, 6)]$
 Retirer : **(12, 3)** et **(8, 4)**
 Insérer le tuple **(B, 7)** dans la liste F
 La liste F triée devient :
 $F = [(A, 4), (29, 6), (B, 7)]$

B**4^{ème} étape**

$F = [(A, 4), (29, 6), (B, 7)]$
 Retirer : **(A, 4)** et **(29, 6)**
 Insérer le tuple **(A, 10)** dans la liste F
 La liste F triée devient :
 $F = [(B, 7), (A, 10)]$

A**5^{ème} étape**

$F = [(B, 7), (A, 10)]$
 Retirer : **(B, 7)** et **(A, 10)**
 Insérer le tuple **(A, 17)** dans la liste F
 La liste F triée devient :
 $F = [(A, 17)]$

A

La liste F ne compte plus qu'un seul élément, le procédé est arrêté.

II. 5- a) Écrire la fonction de complexité linéaire : **insere (F, T)**, qui reçoit en paramètre la liste **F** des fréquences triée dans l'ordre croissant des occurrences, et un tuple **T**, de même nature que les éléments de **F**. La fonction insère le tuple **T** dans **F** de façon à ce que la liste **F** reste triée dans l'ordre croissant des occurrences.

Exemple : $F = [(46, 1), (55, 1), (31, 2), (12, 3), (8, 4), (29, 6)]$ et $T = (17, 5)$.

Après l'appel de la fonction : **insere (F, T)**, la liste **F** devient :

$F = [(46, 1), (55, 1), (31, 2), (12, 3), (8, 4), (17, 5), (29, 6)]$

II. 5- b) Écrire la fonction : **arbre_Huffman (F)**, qui reçoit en paramètre la liste des fréquences **F**, composée des tuples formés des différents éléments de la liste à compresser, et de leurs occurrences. La fonction retourne un arbre binaire de Huffman, crée selon le principe décrit ci-dessus.

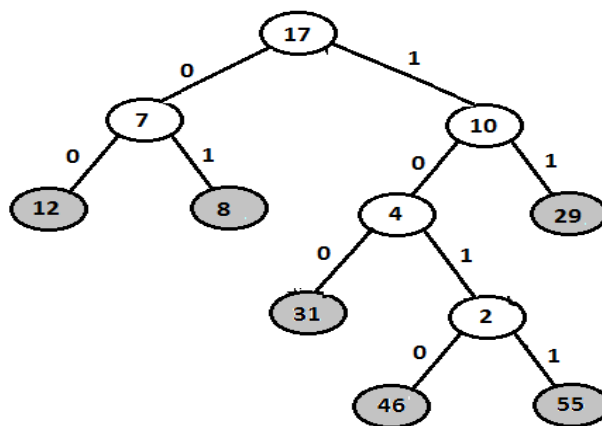
II. 6- Construction du dictionnaire des codes de HUFFMAN :

On suppose que l'arbre binaire de Huffman est créé selon le principe de la question précédente. L'étape suivante est de construire un dictionnaire où les clés sont les différents éléments de la liste à compresser (les feuilles de l'arbre de Huffman), et les valeurs sont les codes binaires de Huffman correspondants.

Parcours de l'arbre binaire de HUFFMAN :

On associe le code **0** à chaque branche de gauche et le code **1** à chaque branche de droite.

Pour obtenir le code binaire de chaque feuille, on parcourt l'arbre à partir de la racine jusqu'aux feuilles en rajoutant à chaque fois au code **0** ou **1** selon la branche suivie :



Ainsi, en parcourant l'arbre, on obtient les codes suivants :

| Feuilles | Codes |
|----------|--------|
| 12 | "00" |
| 8 | "01" |
| 31 | "100" |
| 46 | "1010" |
| 55 | "1011" |
| 29 | "11" |

Écrire la fonction : **codes_Huffman (A, code, codesH)**, qui reçoit trois paramètres :

A : est un arbre binaire de Huffman, créé selon le principe de la question précédente.

code : est une chaîne de caractère initialisée par la chaîne vide.

codesH : est un dictionnaire initialisé par le dictionnaire vide

La fonction effectue un parcours de l'arbre, en ajoutant dans le dictionnaire **codesH**, les feuilles de l'arbre comme clés et les codes binaires correspondants comme valeurs des clés.

Exemple :

codesH = { } # codesH est un dictionnaire vide.

code = " " # code est une chaîne de caractères vide.

Après l'appel de la fonction : **codes_Huffman (Arbre, code, codesH)**, Le contenu du dictionnaire codesH est :

{ 55 : '1011' , 8 : '01' , 12 : '00' , 29 : '11' , 46 : '1010' , 31 : '100' }

II. 7- Compression de la liste des entiers :

Le dictionnaire des codes binaires contient les différents éléments de la liste à compresser, et leurs codes Huffman binaires correspondants. On peut maintenant procéder à la compression de la liste des entiers.

Écrire la fonction : **compresse (L, codesH)**, qui reçoit en paramètres la liste **L** des entiers, et le dictionnaire **codesH** des codes binaires Huffman. La fonction retourne une chaîne de caractères contenant la concaténation des codes binaires Huffman correspondants à chaque élément de **L**.

Exemple :

L = [12 , 29 , 46 , 29 , 31 , 8 , 12 , 55 , 29 , 8 , 12 , 29 , 31 , 29 , 8 , 29 , 8]

codesH = { 55: '1011' , 8: '01' , 12: '00' , 29: '11' , 46: '1010' , 31: '100' }

La fonction **compresse (L, codesH)** retourne la chaîne :

"0011101011100010010111101001110011011101"

II. 8- Décompression de la chaîne de caractères binaires :

L'opération de décompression consiste à retrouver la liste initiale des nombres entiers, à partir de la chaîne binaire et du dictionnaire des codes.

Écrire la fonction : **decompresse (codesH, B)**, qui reçoit en paramètres le dictionnaire **codesH** des codes de Huffman, et la chaîne de caractères binaires **B**. La fonction construit et retourne la liste initiale.

Exemple :

B = "0011101011100010010111101001110011011101"

codesH = { 55: '1011' , 8: '01' , 12: '00' , 29: '11' , 46: '1010' , 31: '100' }

La fonction **decompresse (codesH , B)** retourne la liste initiale :

[12 , 29 , 46 , 29 , 31 , 8 , 12 , 55 , 29 , 8 , 12 , 29 , 31 , 29 , 8 , 29 , 8]

PROBLÈME II : PROGRAMMATION



Remarque 2.1 Dans ce problème il ne sera pas possible d'utiliser les méthodes de la classe *String* suivantes : *capitalize()*, *upper()*, *lower()*, *encode()*, *replace()*, *swapcase()*, *translate()*. Par contre l'utilisation des autres méthodes est possible comme la méthode *indexOf()*.

Nous allons nous intéresser au code de *Vigenère* (1523-1596) qui est plus robuste que le code de *César*, le premier code reconnu comme élément à coder les messages. Nous considérons dans ce problème uniquement l'alphabet de la langue française. On ne s'intéressera qu'aux caractères visibles. On ne prendra pas en compte les ligatures comme par exemple œ ou æ. On ne s'intéresse qu'à des textes composés uniquement de majuscules, les 26 lettres de l'alphabet. Tout autre caractère sera supprimé du texte initial.

Le code de *César* repose sur une substitution mono alphabétique particulière, c'est-à-dire qu'on remplace une lettre de l'alphabet par une autre en utilisant une clé. Chaque lettre de l'alphabet a une valeur entière ($A=0, B=1, C=2, D=3, \dots, K=10, \dots, T=19, \dots, Z=25$) que l'on ajoute à une clé modulo 26 ($x = y \bmod 26 \iff x \equiv y[26]$). Par exemple : si la clé est K (sa valeur est 10, position de la lettre K dans l'alphabet), la lettre T aura comme valeur chiffrée : $19 + 10 \bmod 26 = 29 \bmod 26 = 3$, soit le caractère D.

Le code de *Vigenère* est construit sur le même principe mais il permet de donner une réponse au point faible du code de *César*, à savoir l'analyse des fréquences de l'apparition des lettres. Le code de *Vigenère* utilise des clés plus longues que celui de *César*. Le principe est que chaque lettre peut être codée de plusieurs façons. Par exemple, prenons une clé de dimension 3 (son nombre de lettres). Soit 'ABC' cette clé alors toutes les lettres en position $3k$ seront codées selon le code de *César* avec 'A' comme clé, toutes les lettres en position $3k + 1$ seront codées avec la clé 'B' et toutes les lettres en position $3k + 2$ avec la clé 'C'.

Par exemple : comment chiffrer le texte "c'est un bel été" avec la clé 'ABC' ?

Avant d'appliquer le code de *Vigenère* le texte en clair sera converti en majuscules et ceci sans aucune lettre accentuée ni d'espacement ni autre caractère.

Le texte sera donc transformé en : "CESTUNBELETE".

Appliquons le codage de *Vigenère* : la clé sera répétée autant de fois que nécessaire afin que le texte composé de la répétition globale ou partielle de la clé soit de même taille que le texte initial.

On applique le code de *César* à chaque lettre du texte en clair associée à une lettre de la clé. Par exemple si on prend la deuxième lettre du texte en clair soit 'E' (indice 4 de l'alphabet), associée à la lettre correspondante de la clé, soit 'B' (indice 1), le chiffré est calculé comme :

Texte en clair : CESTUNBELETE
La clé : ABCABCABCABC
Texte crypté : CFUTVPBFNEUG

TABLE 2.1: Un message crypté

$4 + 1 \bmod 26 = 5$, soit la lettre F. On réitère le processus sur l'ensemble des autres lettres du texte en clair. Voir la table 2.1 pour la description du mécanisme.

Première partie

Dans la suite du problème nous utiliserons la phrase suivante définie à l'aide de la variable `texte`.

In [1] : `texte = "Hey Hey, cet été là, le gang des niçois a été arrêté!"`

In [2] : `texte`

Out[3] : 'Hey Hey, cet été là, le gang des niçois a été arrêté!'

Question 8 :

Écrire une fonction ***enMajuscule(CH)*** qui transforme tous les caractères de la chaîne de caractères `CH` passée en tant que paramètre en une chaîne composée uniquement de majuscules. On transformera tous les caractères accentués en majuscules non accentuées. Par exemple le caractère 'ï' sera transformé en caractère 'I'. Tous les autres caractères seront conservés comme par exemple les caractères de ponctuation.

À titre indicatif voici la liste des caractères accentués : âàéèëëïïôùûüÿ. On ajoutera à cette liste le caractère ç (c cédille). Attention on ne pourra pas utiliser la méthode `upper()` de la classe *String* pour traiter cette question et les suivantes.

Exemple :

In [4] : `enMajuscule(texte)`

Out[5] : 'HEY HEY, CET ÉTÉ LA, LE GANG DES NICOIS A ETE ARRETE!'

Question 9 :

Écrire la fonction ***majusculesSeules(CH)*** qui prend une chaîne de caractères `CH`, comme paramètre et retourne une chaîne de caractères composée uniquement des lettres de l'alphabet : ABCDEFGHIJKLMNOPQRSTUVWXYZ.

Exemple :

In [6] : `majusculesSeules(texte)`

Out[7] : 'HEYHEYCETETELALEGANGDESNICOISAETEARRETE'

Question 10 :

Écrire la fonction ***vigenereEncode(CH,CL)*** qui a deux paramètres de type chaîne de caractères, la chaîne de caractères `CH`, à encoder suivant le critère de *Vigenère* et la clé `CL`. Cette fonction retourne la chaîne encodée.

Remarque : la clé sera uniquement composée de lettres de l'alphabet :
ABCDEFGHIJKLMNOPQRSTUVWXYZ.

Exemple :

In [8] : `vigenereEncode(texte, "ABC")`

Out[9] : 'HFAHFACFVEUGLBNEHCNHFETPIDQITCEUGASTEUG'

Question 11 :

Calculer la complexité de la fonction `vigenereEncode()`.

Question 12 :

Écrire la fonction `vigenereEncode()` de manière récursive. On nommera cette fonction `vigenereEncodeRec()` qui comportera autant de paramètres que nécessaire. Vous devez **changer et commenter** les différents paramètres d'entrées que vous allez utiliser.

Pour cette question le texte à traiter ne sera composé que de lettres de l'alphabet :
ABCDEFGHIJKLMNOPQRSTUVWXYZ

Question 13 :

Calculer la complexité de la fonction `vigenereEncodeRec()`.

Question 14 :

Écrire la fonction `vigenereDecode(CH, CL)` qui a deux paramètres de type chaînes de caractères : la chaîne de caractères **CH**, à décoder suivant le critère de **Vigenère** et la clé **CL**. Cette fonction retourne le message en clair.

Exemple :

In [10] : `vigenereDecode("HFAHFACFVEUGLBNEHCNHFETPIDQITCEUGASTEUG", "ABC")`

Out[11] : 'HEYHEYCETETELALEGANGDESNICOISAETEARRETE'

Question 15 :

Indiquer comment on doit choisir la clé afin que le message soit considéré comme robuste à une personne malveillante qui voudrait cracker ce message.

Seconde partie

On s'intéresse maintenant à la manière de casser le code de *Vigenère*, c'est-à-dire à partir du texte crypté de pouvoir déterminer la clé. On peut remarquer que le codage d'une même lettre du texte à crypter avec une lettre se trouvant à une même position de la clé est toujours le même. Ainsi si p est la longueur de la clé utilisée pour coder un texte on peut remarquer que toutes les lettres identiques aux positions $k * l + p$ avec $0 \leq l \leq p - 1$ sont codées par la même lettre pour une valeur l spécifique. Une analyse des fréquences pour les différentes valeurs de l nous permet alors, de deviner la clé utilisée. La table 2.1 donne un exemple d'un message crypté avec une clé de longueur 3. Si la clé est de longueur p , on aura p lettres pour crypter le message. La première lettre du message sera cryptée à l'aide de la première lettre de la clé. La seconde lettre du message sera cryptée à l'aide de la seconde lettre de la clé, et ainsi de suite.

| | | | | | |
|-----|---------|-----------|-----|-----------|-----|
| 0 | p | 2*p | ... | k*p | ... |
| 1 | 1 + p | 1 + 2*p | ... | 1 + k*p | ... |
| 2 | 2 + p | 2 + 2*p | ... | 2 + k*p | ... |
| ... | | | | | |
| i | 1 + p | 1 + 2*p | ... | 1 + k*p | ... |
| ... | | | | | |
| p-1 | p-1 + p | p-1 + 2*p | ... | p-1 + k*p | ... |

TABLE 2.2: Mécanisme de cryptage

Tous les caractères du message distants de p caractères utiliseront une même lettre de la clef pour réaliser le chiffrement. La table 2.2 présente le mécanisme d'un tel chiffrement. Il est possible d'utiliser p lettres différentes pour coder chacune des séquences, cela dépend de la nature de la clé.

Afin de simplifier le problème nous considérerons, par la suite, que la dimension de la clé est connue. Nous allons mettre en œuvre une méthode relativement simple qui ne fonctionnera que dans un cadre bien spécifique. Voici un message crypté dont vous devez retrouver la clé : message = 'HFAHFACFVEUGLBNEHCNHFETPIDQITCEUGASTEUG'

Question 16 :

Écrire une fonction **sousChaines(CH,d,p)** qui prendra en compte trois paramètres : **CH** le texte à déchiffrer, **d** le déplacement (c'est-à-dire la lettre utilisée de la clé) et **p** la dimension de la clé. Cette fonction retourne une chaîne de caractères. Par exemple : à l'indice d de la clé on va extraire du texte chiffré tous les caractères qui sont espacés d'une distance p , la dimension de la clé.

Autrement dit cette fonction rend la sous chaîne composée des caractères du texte aux indices $d, d + p, d + 2p$, etc.

Exemple :

In [12] : sousChaines(message, 1,3)

Out[13] : 'FFUBHHTDTUSU'

Question 17 :

Écrire la fonction **listeDesSousChaines(CH,d)** qui prendra en compte deux paramètres, **CH** le texte crypté et **d** la dimension de la clé. Cette fonction retourne comme valeur un tableau avec l'ensemble de toutes les sous-chaines définies à la question précédente.

Exemple :

In [14] : listeDesSousChaines(message, 3)

Out[15] : ['HCELENEIEAE', 'FFUBHHTDTUSU', 'AAVGNCFPQCGTG']

Question 18 :

C'est le caractère 'E' qui apparaît le plus souvent dans un texte français. Les caractères suivants, les plus fréquents, sont dans l'ordre ['A', 'S', 'I', 'T', 'N']. Nous nous limiterons uniquement au caractère 'E'. Dans chaque sous-chaine de caractères constituée à la question précédente

il suffit de connaître la fréquence des caractères afin de déterminer quel est le caractère qui apparaît le plus. Chacun de ces caractères correspond à un chiffrement déterminé à partir du caractère 'E'.

Écrire la fonction **frequenceCaracteres(CH)** qui prend comme paramètre un texte crypté **CH** et retourne comme valeur un tableau constitué des fréquences de chaque caractère de l'alphabet de ce texte.

Exemple :

In [16] : `frequenceCaracteres(message)`

Out[17] : [3,1,3,1,5,4,3,4,2,0,0,1,0,2,0,1,1,0,1,3,3,1,0,0,0,0]

Question 19 :

À l'aide de la question précédente écrire une fonction qui permettra l'extraction de la clé. Afin de simplifier le traitement on fera référence uniquement à la lettre 'E' comme le caractère qui apparaît le plus dans le texte en clair.

Cette fonction que l'on nomme **code(CH,p)** a deux paramètres : **CH** le texte crypté et **p** la longueur de la clé. cette fonction retourne une chaine de caractères, la valeur de la clé.

Exemple :

In [18] : `code(message, 3)`

Out[19] : 'ABC'

On retrouve bien la clé initiale à partir du texte crypté.

Épreuve d'informatique – Session 2015 – Filière MP/ PSI/ TSI

Problème : Gestion des expressions postfixées

Remarque :

Les deux parties de ce problème sont indépendantes. La deuxième partie peut être faite en prenant les résultats de la première partie.

Par définition une expression est une suite de nombres associée à un ensemble d'opérateurs avec éventuellement un jeu de parenthèses ouvrantes et fermantes.

L'objet de ce problème est de gérer des expressions postfixées, expressions algébriques constituées d'éléments appelés opérandes et d'opérateurs. La position de l'opérateur par rapport aux opérandes indique le type d'expression algébrique : infixe, préfixée ou postfixée. La notation préfixée est souvent appelée notation polonaise car inventée par le mathématicien Jan Lukasiewicz et par opposition la notation postfixée est appelée notation polonaise inverse.

Il n'y aura aucune priorité particulière entre les différents opérateurs.

Exemple :

Notation infixe : $(1+5) * 6$

Notation préfixée : $+ 1 6 * 5$

Notation postfixée : $1 5 + 6 *$

Autre exemple :

L'écriture postfixée de $(3+2) * 5$ est $3 2 + 5 *$ et celle de $3+(2 * 5)$ est $3 2 5 * +$.

Question 9 :

Citer un intérêt d'utiliser la notation postfixée plutôt que la notation infixe.

Première partie

Afin de réaliser le traitement, c'est-à-dire le calcul d'une expression postfixée nous avons besoin d'un composant, une file d'attente, une "pile" de type FIFO (First In First Out).

Par définition une pile sera implémentée à l'aide d'une liste, le sommet de la pile sera toujours situé en début de liste. Vous devez dans cette première partie définir différentes fonctions capables d'utiliser une telle file d'attente qui n'aura pas de limite.

Afin que la pile puisse fonctionner correctement il faudra vérifier lors de la sortie d'un élément, que la file n'est pas vide. L'exception **PileVideException** est définie par défaut.

Pour l'implémentation de votre **Pile** vous devez utiliser les fonctions fournies dans l'annexe.

Question 10 :

Écrire une fonction **initPile()** afin d'initialiser une pile.

Question 11 :

Écrire la fonction **estVide(pile)** qui indique si une pile est vide.

Question 12 :

Écrire la fonction **empiler(pile, elem)** qui ajoutera à la file d'attente l'élément *elem*.

Question 13 :

Écrire la fonction **depiler(pile)** qui supprimera de la file d'attente le dernier élément entré et retournera cet élément.

Question 14 :

Écrire la fonction **valeurSommet(pile)** qui affichera l'élément se trouvant au sommet de la file d'attente. Cette fonction ne supprimera pas cet élément.

Question 15 :

Écrire la fonction **hauteur(pile)** qui affichera le nombre d'éléments se trouvant dans la pile.

Deuxième partie :

Principe de l'évaluation. Pour calculer une expression arithmétique codée en postfixée, il suffit :

- d'empiler les entiers successifs que l'on rencontre ;
- lorsque l'on détecte un opérateur, il faut
 - dépiler les 2 derniers entiers ;
 - leur appliquer l'opérateur ;
 - empiler le résultat ;
- enfin, lorsque l'on rencontre un point, le résultat du calcul doit être dépilé puis affiché.

Question 16 :

On souhaite tester si une expression est bien un nombre. La fonction **estEntier()** donnera comme résultat une valeur booléenne : True si l'expression est un nombre entier et False sinon. Attention, il n'est pas possible d'utiliser la fonction **isinstance()**. La fonction **estEntier()** sera de style récursif.

```
>>>estEntier('12')
True
>>>estEntier(12)
True
>>>estEntier('1a4')
False
```

Question 17 :

Définir la fonction **eval()** qui sera constituée de trois paramètres, un opérateur (un caractère), suivi de deux opérandes (deux nombres entiers). Deux exceptions sont définies par défaut. L'exception **OpNonValideException** sera levée lorsqu'un opérateur ne sera pas valide et l'exception **ArgNonValideException** si un opérande est non valide. La liste des opérateurs utilisés est la suivante : +, -, *, /, . (le caractère point).

```
>>>eval('+', 1, 2)
3
>>>eval('+', '1', 2)
...
ArgNonValideException
```

Question 18 :

Définir la fonction **evaluate()** qui aura comme arguments, l'expression postfixée à analyser. Cette fonction retournera la valeur du calcul correspondant à cette expression.

```
>>>evaluate([7, 4, '+', '.'])
11
```

Question 19 :

On s'intéresse à présent aux expressions infixées, comme par exemple : $1+3*4$ qui sera traduite par l'expression ['1','+', '3','*', '4','.']. Les parenthèses sont souvent utilisées afin de donner le sens que l'on souhaite aux différents calculs. Par exemple $(1+3)*4$ et $1+(3*4)$ donneront deux résultats différents. Dans cette question une expression pourra utiliser différents niveaux d'imbrication de parenthèses, comme par exemple $(1+(5*(6+2)))$.

Définir la fonction **estBienParenthesee()** qui aura une expression (une liste de caractères) comme paramètre et retournera comme valeur une chaîne de caractère *Expression mal parenthésée* ou *Expression bien parenthésée*.

Question 20 :

Dans cette question les expressions infixées n'auront pas de parenthèses.

Définir la fonction **transInfix()** qui aura comme paramètre une expression infixée qui retournera une expression postfixée. On mettra en œuvre un mécanisme d'exception pour tester si l'expression fournie est vide ou mal formée.

```
>>>transInfix(['1', '+', '3', '*', '4', '+', '8', '.'])
['8', '4', '+', '3', '*', '1', '+', '.']
```

Question 21 :

En se limitant à un seul niveau de parenthèses, expliquez comment vous feriez pour transformer une expression infixée en une expression postfixée. On ne demande pas du code mais le mécanisme à mettre en œuvre.

```
>>>transInfix1SeulNiveau(['(', '1', '+', '2', ')', '*', '4', '.'])
['1', '2', '+', '4', '*', '.']
```

Question 22 :

Il y a différentes manières d'exprimer une expression postfixée. Les deux expressions suivantes sont totalement identiques, forme1 : ['8', '4', '+', '3', '*', '1', '-', '.'] et

forme2 : ['1', '3', '4', '8', '+', '*', '-', '.']

Définir la fonction **transTotal()** qui transforme une expression postfixée de type forme1 en une expression postfixée de type forme2.

```
>>>transTotal(expr)
['1', '3', '4', '8', '+', '*', '+', '.']
```

Question 23 :

Modifier la fonction précédente afin que le paramètre passé à **transTotal()** soit non pas une expression postfixée de type 1 mais une expression infixée.

```
>>>transTotal(['1', '+', '3', '*', '4', '+', '8', '.'])
['8', '4', '3', '1', '+', '*', '+', '.']
```